

Perfilado de actividad de usuario en entornos NAS mediante análisis de trazas SMB2 con máquinas de estados finitas



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor: Mario Vicente Obeso

Director: Aitor Eguzkitza Diego

Pamplona, junio de 2026

ÍNDICE GENERAL

RESUMEN.....	3
LISTA DE PALABRAS CLAVE.....	4
ABSTRACT.....	5
KEYWORDS.....	5
ALINEACIÓN CON LOS OBJETIVOS DE DESARROLLO SOSTENIBLE (ODS).....	6
ODS 9: Industria, Innovación e Infraestructura.....	6
ODS 8: Trabajo Decente y Crecimiento Económico.....	6
ODS 16: Paz, Justicia e Instituciones Sólidas.....	6
1. INTRODUCCIÓN Y OBJETIVOS.....	7
1.1. Introducción y Motivación.....	7
1.2. Objetivos del Trabajo.....	7
1.3. Alcance del Proyecto (Scope).....	8
2. ESTADO DEL ARTE Y FUNDAMENTOS TÉCNICOS.....	9
BLOQUE I: Contexto, Origen y Alcance.....	9
BLOQUE II: Análisis de Sistemas de Almacenamiento y Soluciones de Auditoría.....	10
BLOQUE III. Fundamentos de SMB2 para el Análisis de Trazas.....	11
3. METODOLOGÍA.....	13
1. Análisis de trazas SMB2 y definición de operaciones de usuario.....	13
4. DESARROLLO Y EXPERIMENTACIÓN.....	31
1. Introduccion.....	31
2. Evolucion del analizador.....	32
3. Analizador v4: Operaciones simples y compuestas.....	34
4. Clasificacion de operaciones.....	37
5. Problemas encontrados durante el desarrollo.....	40
6. Validacion y pruebas.....	42
7. Resultados con trazas reales.....	44
5. CONCLUSIONES Y LÍNEAS FUTURAS.....	48
1. Conclusiones.....	48
2. Limitaciones.....	50
3. Trabajo Futuro.....	52
6. BIBLIOGRAFÍA.....	54
ANEXOS.....	55
AGRADECIMIENTOS.....	56

RESUMEN

La infraestructura de almacenamiento corporativo ha migrado mayoritariamente hacia arquitecturas centralizadas basadas en servidores de archivos y cabinas NAS (*Network Attached Storage*). En este entorno, el protocolo **SMB2** (*Server Message Block 2*) constituye el estándar de comunicación en redes Windows y permite que cada acción del usuario sobre el sistema de archivos apertura, escritura, borrado, renombrado— genere una secuencia de paquetes de red directamente observable y estructurada. Sin embargo, cuando la misma actividad se produce a través de servicios cloud (OneDrive, Google Drive), el tráfico viaja cifrado mediante protocolos propietarios, lo que crea un punto ciego para el administrador sin recurrir a herramientas intrusivas instaladas en el equipo cliente.

El presente Trabajo de Fin de Grado desarrolla un **analizador de tráfico SMB2** capaz de transformar trazas de red en bruto, capturadas de forma completamente pasiva con Wireshark, en un registro legible de operaciones de usuario de alto nivel: subida, descarga, copia, borrado, renombrado, compresión y modificación de archivos y carpetas. La herramienta no inspecciona el contenido de los ficheros (*payload*), preservando así la privacidad, y no requiere la instalación de agentes en los equipos cliente ni interfiere en el rendimiento de la red.

La metodología se apoya en tres pilares técnicos. En primer lugar, el **análisis semántico del protocolo SMB2**: los campos de cabecera (*CreateOptions*, *DesiredAccess*, *FileInformationClass*, *NT_Status*) y el identificador único *FileID* (GUID de 128 bits) permiten reconstruir la secuencia lógica de cada operación sin ambigüedad. En segundo lugar, el diseño de **máquinas de estados finitas (FSM)** que modelan cómo los comandos atómicos del protocolo (CREATE, READ, WRITE, SET_INFO, CLOSE) se encadenan para representar acciones de usuario coherentes. En tercer lugar, un **algoritmo de agrupación jerárquica con pre-escaneo de FileIDs**, que resuelve la asincronía inherente a SMB2 —donde el cliente puede enviar múltiples solicitudes sin esperar respuesta— y permite detectar operaciones compuestas que involucran múltiples FileIDs simultáneos (copias de carpetas, compresiones, modificaciones con editor).

El analizador ha evolucionado a través de cuatro versiones iterativas. La versión final (v4) implementa un clasificador de dos niveles: 21 reglas en cascada para operaciones atómicas (un único FileID) y 7 reglas multi-FileID para operaciones compuestas. La validación se realizó sobre tres trazas reales: una traza de usuario de 26.817 paquetes, una traza de ransomware de 26.742 paquetes y una traza corporativa de una semana de actividad con 17,18 millones de paquetes. Los resultados demuestran una tasa de clasificación del 99,1% en la traza de usuario y permiten identificar patrones característicos del comportamiento del ransomware —creación masiva de archivos vacíos, borrado y renombrado sistemático— diferenciándolos del tráfico legítimo.

El trabajo concluye que el análisis pasivo del protocolo SMB2 proporciona una visibilidad forense completa sobre la actividad de los usuarios en entornos de almacenamiento centralizado, constituyendo una base técnica sólida para el desarrollo futuro de sistemas de detección de intrusiones y clasificadores binarios ransomware/usuario basados en el perfil de operaciones observado.

LISTA DE PALABRAS CLAVE

SMB2 (Server Message Block 2) · Análisis de tráfico de red · Máquinas de estados finitas (FSM) ·
Wireshark · Python · Ransomware · Detección de anomalías · Auditoría no intrusiva ·
Almacenamiento en red (NAS) · Perfilado de usuario

ABSTRACT

Corporate file storage infrastructure has largely migrated to centralized architectures based on file servers and NAS systems. In this context, the SMB2 (Server Message Block 2) protocol is the standard for Windows-based networks, making every user action on the file system — opening, writing, deleting, renaming — observable as a structured sequence of network packets. However, when the same activity occurs through cloud storage services (OneDrive, Google Drive), traffic is encrypted using proprietary protocols, creating a blind spot that cannot be resolved without deploying intrusive endpoint agents.

This Final Degree Project develops an SMB2 traffic analyzer capable of transforming raw network traces — captured passively with Wireshark — into a human-readable log of high-level user operations: file and folder uploads, downloads, copies, deletions, renaming, compression, and modification. The tool does not inspect file payload, preserving user privacy, and requires neither client-side agent installation nor any impact on network performance.

The methodology relies on three technical pillars: (1) semantic analysis of SMB2 header fields (CreateOptions, DesiredAccess, FileInformationClass, NT_Status) and the 128-bit FileID GUID to reconstruct operation sequences unambiguously; (2) finite state machine (FSM) models that capture how atomic protocol commands chain into coherent user actions; and (3) a hierarchical grouping algorithm with FileID pre-scanning that handles SMB2's inherent asynchrony and detects compound operations involving multiple simultaneous FileIDs (folder copies, compressions, editor-based modifications). Validation against three real-world traces — a 26,817-packet user trace, a 26,742-packet ransomware trace, and a one-week corporate trace of 17.18 million packets — demonstrates 99.1% classification accuracy and clear behavioral differentiation between legitimate and ransomware traffic.

The project concludes that passive analysis of the SMB2 protocol provides complete forensic visibility over user activity in centralized storage environments, establishing a solid technical foundation for the future development of intrusion detection systems and binary ransomware/user classifiers based on the observed operation profile.

KEYWORDS

SMB2 (Server Message Block 2) · Network traffic analysis · Finite state machines (FSM) · Wireshark · Python · Ransomware detection · Anomaly detection · Non-intrusive auditing · Network-attached storage (NAS) · User profiling

ALINEACIÓN CON LOS OBJETIVOS DE DESARROLLO SOSTENIBLE (ODS)

El presente Trabajo de Fin de Grado se alinea con la Agenda 2030 de las Naciones Unidas, contribuyendo directamente a los siguientes objetivos mediante la mejora de la seguridad y la eficiencia en las infraestructuras digitales:

ODS 9: Industria, Innovación e Infraestructura

Es el objetivo principal de este trabajo. Al desarrollar una metodología de auditoría no intrusiva basada en el protocolo SMB2, se está fortaleciendo la **resiliencia de las infraestructuras de datos**.

- **Justificación:** El proyecto propone una innovación técnica para la monitorización de cabinas de datos, lo que permite a las organizaciones proteger sus activos de información frente a ataques (como el Ransomware), garantizando una infraestructura tecnológica más robusta y fiable.

ODS 8: Trabajo Decente y Crecimiento Económico

La ciberseguridad es un pilar fundamental para la estabilidad económica.

- **Justificación:** Un entorno de almacenamiento seguro previene la pérdida de datos y el espionaje industrial, factores que pueden comprometer la viabilidad de las empresas y el empleo. Al mejorar la visibilidad sobre las acciones de los usuarios, se protege el capital intelectual y la continuidad del negocio en el entorno digital.

ODS 16: Paz, Justicia e Instituciones Sólidas

El presente trabajo contribuye a las metas de este objetivo enfocadas en la reducción de la delincuencia tecnológica y el fortalecimiento de la transparencia institucional mediante la seguridad digital.

- **Justificación:** La metodología desarrollada permite la reconstrucción forense y cronológica de las acciones de un usuario sobre el sistema de archivos de manera completamente pasiva y no intrusiva. Al decodificar identificadores críticos del protocolo SMB2 (como *SessionID* y *FileID*), la herramienta provee una trazabilidad matemática exacta de "quién ejecutó qué operación y en qué instante", garantizando principios de no repudio y rendición de cuentas (*accountability*) dentro de la red corporativa. Esta visibilidad es un pilar tecnológico clave para la detección temprana del cibercrimen —especialmente ataques automatizados como el *ransomware*— y para mitigar el acceso no autorizado o la exfiltración de información sensible.

1. INTRODUCCIÓN Y OBJETIVOS

1.1. Introducción y Motivación

Hoy en día, el almacenamiento de datos ya no se gestiona de forma local en cada equipo, sino que se centraliza en servidores de archivos y cabinas NAS (Network Attached Storage). Esta estructura es la base de la mayoría de las empresas, pero tiene un problema concreto: si no se vigila el tráfico de red de forma específica, no sabemos realmente qué ocurre con los datos.

La motivación de este proyecto nace de una limitación de los servicios cloud actuales como OneDrive o Google Drive. Estas plataformas usan protocolos cerrados y cifran el tráfico de principio a fin. Para un administrador de red, esto crea un punto ciego: es imposible saber desde la red si un usuario está borrando, moviendo o exfiltrando información sin acceder directamente a su equipo.

El protocolo SMB2, en cambio, es el estándar de comunicación en redes locales Windows y es mucho más transparente. Este trabajo parte de la idea de que, analizando los paquetes SMB2, se puede reconstruir con precisión qué está haciendo el usuario — sin necesidad de instalar software en su equipo ni de forzar el descifrado del tráfico cloud. El objetivo es usar esa información para monitorizar la actividad diaria y, sobre todo, detectar comportamientos peligrosos como los que caracterizan a un ataque de ransomware.

1.2. Objetivos del Trabajo

Objetivo principal

El propósito central de este Trabajo de Fin de Grado es el **estudio, análisis y caracterización técnica del tráfico SMB2** en entornos de almacenamiento centralizado. El fin último es desarrollar un modelo analítico capaz de transformar ráfagas de datos binarios y comandos atómicos en un registro de actividad de usuario legible, proporcionando visibilidad total sobre las operaciones que los usuarios ejecutan contra la cabina de datos.

Objetivos específicos

- **Análisis de protocolo e inspección de metadatos:** Investigar y documentar cómo los campos de control (flags) y códigos de operación de SMB2 revelan la actividad del usuario. El objetivo es **identificar operaciones del sistema de archivos** (lectura, escritura, borrado o renombrado) mediante el análisis de trazas de red basadas en los mensajes del protocolo, sin inspeccionar el contenido de los ficheros (payload) para preservar la privacidad.
- **Diseño de máquinas de estados finitas (FSM):** Modelar las operaciones del sistema de archivos mediante una arquitectura de estados lógicos que consolide eventos dispersos (una serie de peticiones **WRITE** seguida de un **CLOSE** final), por ejemplo, en unidades de acción lógicas y auditables para un analista.
- **Caracterización de comportamiento y patrones de tráfico:** Establecer una base métrica de huellas de red que defina el comportamiento normal del usuario, considerando rítmica,

volúmenes de datos y tipos de recursos accedidos. Esa referencia permite distinguir el uso legítimo de patrones automatizados o maliciosos.

- **Filtrado de ruido:** Desarrollar criterios de discriminación técnica para separar las acciones humanas del tráfico de mantenimiento generado por el sistema operativo: encuestas de estado, previsualizaciones de iconos y tráfico similar que contamina las trazas.

1.3. Alcance del Proyecto (Scope)

El alcance de este proyecto se delimita de la siguiente manera:

- **Protocolo:** El análisis técnico se restringe a la versión **SMB 2.0**. Se selecciona esta versión por ser el estándar en la industria actual, dejando fuera la versión 1.0 por estar en desuso y presentar vulnerabilidades críticas conocidas.
- **Metodología de Captura:** El proyecto se basa en el análisis de trazas reales (*pcap*) capturadas de forma no intrusiva. La solución no debe interferir en la latencia de la red ni requerir la instalación de agentes en los equipos clientes, preservando la transparencia del sistema.
- **Nivel de Salida Analítica:** El trabajo concluye en la fase de **generación de perfiles de usuario**. Aunque los resultados permiten identificar comportamientos sospechosos, la implementación de mecanismos de respuesta activa (bloqueo de IPs o cierre de sesiones) se considera fuera del alcance, centrándose la investigación en la capacidad de detección y visibilidad forense.

1.4. Disponibilidad del Código Fuente y Repositorio

Como parte fundamental de las aportaciones prácticas de este Trabajo Fin de Grado, **el autor ha diseñado, programado y validado de forma íntegra** la herramienta objeto de estudio. Con el fin de garantizar la transparencia, la reproducibilidad de los experimentos y permitir la libre auditoría del software por parte del tribunal, el código fuente completo del analizador **SMB2-OpAnalyzer**, los módulos de las máquinas de estados (FSM) y la batería de pruebas automatizadas se encuentran alojados en el repositorio público de GitHub del autor:

- **Repositorio oficial del proyecto:**
<https://github.com/vicente154000/smb2-cloud-traffic-analyzer> [13]

La arquitectura interna del script, el modelo de datos de los diccionarios de contexto y la guía de despliegue para su ejecución directa se detallan de manera extendida en el **Anexo D** de esta memoria.

2. ESTADO DEL ARTE Y FUNDAMENTOS TÉCNICOS

BLOQUE I: Contexto, Origen y Alcance

1. El Ecosistema de Almacenamiento y el Agente de Sincronización

El almacenamiento de datos en las empresas ha migrado progresivamente hacia infraestructuras centralizadas donde los usuarios acceden a sus archivos a través de la red. En este contexto, el protocolo SMB2 es el estándar de comunicación en entornos Windows y en la mayoría de cabinas NAS. Cada acción del usuario sobre el sistema de archivos —abrir, guardar, borrar, mover— genera una secuencia de paquetes SMB2 directamente observable en el tráfico de red, sin intermediarios. El problema aparece cuando esa actividad se produce a través de servicios cloud: el tráfico viaja cifrado con TLS mediante protocolos propietarios, lo que crea un punto ciego para el administrador de red. Desde fuera es imposible saber qué está ocurriendo con los datos sin acceder directamente al equipo del usuario.

2. Evolución del Objetivo y Alcance

Este proyecto parte precisamente de esa limitación. El análisis del tráfico cloud quedó descartado desde las fases iniciales al comprobar que los protocolos de sincronización no están documentados y que el cifrado con certificados propios del proveedor hace inviable cualquier inspección externa sin recurrir a ingeniería inversa. SMB2, en cambio, es un protocolo abierto donde la relación entre la acción del usuario y el paquete de red es directa y verificable. Concentrar el esfuerzo en SMB2 tiene además una lógica de continuidad: entender con precisión cómo el protocolo representa que un usuario crea, mueve o borra un archivo es el paso previo necesario para identificar esos mismos patrones cuando aparecen en entornos más opacos.

3. El Protocolo SMB2 como Fuente de Datos

SMB2 opera en la capa de aplicación del modelo OSI y expone directamente la semántica de cada operación. Un renombrado se identifica por un **SET_INFO** con **FileRenameInformation** [1, 2.2.39]; una creación de carpeta se distingue de la creación de un archivo por el bit **FILE_DIRECTORY_FILE** en el campo **CreateOptions** [1, 2.2.13]; un borrado se detecta por el flag **DeletePending** antes del cierre del handle. Esta riqueza de detalles, combinada con el modelo petición-respuesta del protocolo, permite reconstruir con exactitud qué hizo el usuario, sobre qué archivo y en qué momento, sin inspeccionar el contenido de los ficheros. Es esa capacidad de traducir tráfico de red en actividad de usuario lo que convierte a SMB2 en la base de este trabajo.

BLOQUE II: Análisis de Sistemas de Almacenamiento y Soluciones de Auditoría

1. Arquitecturas basadas en host (servicios cloud)

Cuando el almacenamiento opera a través de servicios cloud, la auditoría del tráfico de red deja de ser viable. El agente de sincronización que reside en el equipo del usuario actúa como intermediario: decide cuándo enviar los datos, los procesa localmente, los cifra con TLS y los transmite mediante protocolos propietarios no documentados. El resultado desde el punto de vista del análisis de red es siempre el mismo: un flujo de bytes opacos del que no se puede extraer ninguna información sobre la actividad real del usuario. No es posible distinguir si ese tráfico corresponde a una subida, un borrado o una simple actualización de metadatos.

Auditar este modelo requiere acceder directamente al terminal mediante agentes de telemetría (EDR/DLP), con todo lo que eso implica: necesidad de permisos administrativos en cada equipo, dependencia de logs internos que pueden ser incompletos o manipulados, y una visión de la actividad que siempre llega con el desfase que introduce el propio agente de sincronización. Es un enfoque intrusivo que, además, no escala bien en entornos con un número elevado de equipos.

2. Arquitecturas basadas en red (NAS y servidores de archivos)

En infraestructuras que usan SMB2, toda interacción con los datos pasa obligatoriamente por el puerto 445. Monitorizar ese único punto ofrece una visión completa y pasiva de la actividad de todos los usuarios sin necesidad de instalar nada en los equipos cliente ni solicitar permisos adicionales. Es un enfoque no intrusivo que no afecta al rendimiento de la red ni a la experiencia del usuario.

A diferencia del modelo cloud, no existe capa de abstracción entre la acción del usuario y el paquete de red: cuando el usuario borra un archivo se genera en ese instante un **SET_INFO** con **DeletePending**; cuando lo mueve arranca una secuencia **CREATE** → **SET_INFO** → **CLOSE**. Esa inmediatez permite auditar en tiempo real, confirmar el resultado de cada operación a través del campo **NT_Status**, y reconstruir secuencias completas de actividad agrupando paquetes por su **FileID**, un identificador único de 128 bits que vincula todos los comandos sobre un archivo desde su apertura hasta su cierre.

3. Comparativa de soluciones y capacidad de análisis

La diferencia entre ambos enfoques se reduce a una cuestión de visibilidad y de coste operativo. En el modelo cloud, el tráfico es asíncrono, cifrado y opaco; la auditoría requiere acceso al terminal, produce logs internos de fiabilidad variable y no permite reconstruir la actividad desde la red. En el modelo SMB2, el tráfico es estructurado y abierto, la captura es no intrusiva desde un único punto de la red, y el análisis puede llegar al nivel de operación individual sobre un archivo concreto, con confirmación de éxito incluida. La tabla comparativa completa entre ambas arquitecturas se recoge en el Anexo A.

BLOQUE III. Fundamentos de SMB2 para el Análisis de Trazas

Para construir una metodología capaz de identificar operaciones de usuario a partir de trazas de red, es imperativo profundizar en los mecanismos internos del protocolo establecidos en la documentación técnica de referencia **MS-SMB2 [1]**. Este bloque detalla los pilares sobre los cuales se asienta la lógica de este proyecto, permitiendo elevar el análisis desde paquetes aislados hacia unidades de información semántica.

1. Arquitectura de Cabeceras y Comandos Atómicos

Cada paquete SMB2 comienza con una cabecera fija de 64 bytes que contiene la información de control necesaria para que el cliente y el servidor se entiendan. Para los objetivos de esta investigación, los campos más relevantes son:

Comandos de Conexión y Gestión de Sesión

Estos comandos son los que preparan el escenario. Sin ellos, no hay FileID ni intercambio de datos. Las tablas de comandos de conexión completas se recogen en el **Anexo A** (comandos de conexión).

Una vez establecida la sesión, la interacción con el sistema de archivos se descompone en los siguientes comandos recogidos en el **Anexo A** (comandos atómicos)

2. Identificadores de Seguimiento y Contexto

Para agrupar los comandos dispersos en una red con tráfico masivo, el protocolo emplea una jerarquía de identificadores que vinculan cada paquete a una acción lógica, todo esto se recoge en [1, 2.2.1.2 SMB2 Packet Header]:

- **MessageID:** Es un número de secuencia incremental que permite emparejar cada petición (REQ) con su respuesta (RSP). Es vital para calcular tiempos de respuesta y manejar el desorden de paquetes en entornos reales .
- **SessionID:** Vincula los paquetes a una conexión TCP y sesión de usuario específica, permitiendo aislar el tráfico de diferentes clientes.
- **TreeID (Identificador de Recurso Compartido):** Este ID es asignado por el servidor en el momento en que el cliente se conecta a un recurso compartido específico (mediante un TREE_CONNECT). Su función es identificar qué volumen o carpeta compartida concreta se está utilizando dentro de una sesión. Como ventaja analítica, el TreeID permite que un usuario mantenga una misma sesión a nivel de aplicación aunque la conexión TCP subyacente cambie, garantizando la continuidad del rastro.
- **FileID (GUID):** Es el identificador de 128 bits generado tras un CREATE exitoso [1, 2.2.14.1 SMB2_FILEID]. Este ID es el "hilo conductor" que permite asociar múltiples comandos (Reads, Writes, Closes) a un único archivo o carpeta, sirviendo de base para nuestras máquinas de estados.

Nota sobre la persistencia: El FileID solo es válido mientras el manejador (*handle*) del archivo permanezca abierto. Esto lo convierte en la herramienta perfecta para reconstruir

la secuencia lógica de una operación (apertura, ejecución y cierre). Sin embargo, al no ser persistente, no permite vincular por sí solo dos aperturas distintas del mismo fichero; para esa asociación se requiere el análisis del *Filename*.

3. Atributos de Control y Flags de Intencionalidad

El análisis pasivo del protocolo SMB2 se fundamenta en la inspección de metadatos críticos embebidos en las cabeceras de las solicitudes y respuestas del cliente. Campos específicos como **CreateOptions** (para la discriminación de tipos de objetos), **FileInformationClass** (para la naturaleza de los metadatos), **DesiredAccess** [1, 2.2.13] (que revela la intencionalidad de la acción) junto con las métricas de **Length**, **Offset** y el código de estado **NTSTATUS** [9, 2.3.1 **NTSTATUS Values**], constituyen el núcleo de variables que alimentan el motor de clasificación. El desglose técnico, la semántica y los valores operativos de estos atributos de control se detallan en el **Anexo A**.

4. Dimensión Temporal y Análisis Rítmico

Para que la transformación de datos técnicos en flujos de actividad sea precisa, la metodología implementada se apoya en un análisis proactivo del tráfico de red, permitiendo asegurar la fiabilidad y la detección de anomalías en la comunicación [3]. Este análisis se desglosa en dos ejes:

- **Timestamps (Epoch y Relativo):** Se capturan los tiempos de inicio y fin de cada ciclo **Request/Response**. El registro preciso de estos sellos temporales permite calcular no solo el tiempo de respuesta de la infraestructura, sino también el **ritmo operacional**. Este cálculo es esencial para reconstruir la secuencia lógica de las operaciones de archivo definidas en el estándar [1].
- **Caracterización del Comportamiento:** Siguiendo los modelos de predicción de tráfico [3], se establece una distinción entre tipos de tráfico según su cadencia. Un flujo constante de ráfagas a alta velocidad suele indicar procesos automatizados (como *scripts*, *backups* o la actividad propia de los agentes de sincronización), mientras que los intervalos irregulares y los tiempos de reflexión (*think time*) entre comandos caracterizan la interacción humana directa sobre el sistema de archivos.

3. METODOLOGÍA

1. Análisis de trazas SMB2 y definición de operaciones de usuario

1.1 Fundamentos Técnicos para el Análisis de Trazas SMB2

El análisis de trazas SMB2 se apoya en un conjunto de identificadores y campos de control que permiten reconstruir la actividad del usuario a partir de los paquetes capturados. El más importante es el **FileID**, un GUID de 128 bits que el servidor asigna tras cada **CREATE** exitoso y que vincula todos los comandos posteriores sobre ese archivo hasta su **CLOSE**. El **MessageID** complementa este mecanismo emparejando cada petición con su respuesta, lo que permite conocer tanto el nombre del archivo (presente solo en el Request) como el resultado de la operación (presente solo en el Response). Para filtrar el tráfico de mantenimiento de sesión [1, 2.2.1.1 SMB2 Packet Header - ASYNC] —**NEGOTIATE**, **SESSION_SETUP**, **ECHO**, **TREE_CONNECT**— y quedarse exclusivamente con las operaciones de archivo, el análisis se apoya en el campo **smb2.nt_status == 0x0** [4], que descarta cualquier operación fallida, y en el campo **CreateDisposition**, que permite distinguir una creación real de una simple apertura o sobrescritura. La descripción detallada de estos campos, sus valores posibles y los filtros de Wireshark empleados se recogen en el **Anexo B (Apartados 1 al 5)**.

1.2 Análisis del comando FIND y la recursividad

El comando que herramientas como Wireshark identifican como FIND corresponde técnicamente al código de operación 0x0E (SMB2 QUERY_DIRECTORY) [1]. Su función principal es obtener un listado de los objetos (archivos y subcarpetas) contenidos dentro de un directorio abierto previamente, usando como patrón de búsqueda generalmente el asterisco (*) para recuperar todo el contenido.

En el marco de esta metodología, el análisis se centra específicamente en el mensaje de respuesta (**FIND Response**) por su valor informativo superior frente a la petición:

- **Confirmación de contenido:** el Request solo indica la intención de buscar; el Response contiene la carga útil con el listado real de archivos y carpetas.
- **Validación de éxito:** sólo a través de la respuesta se puede verificar, mediante el campo Status, si la operación se completó con éxito (STATUS_SUCCESS) o si la carpeta estaba vacía o el acceso fue denegado .

Identificación de operaciones

El comando FIND actúa como el componente determinante para distinguir entre operaciones simples y recursivas sobre directorios. Su ejecución es estructuralmente necesaria en las siguientes categorías:

- **Consultas de estructura:** Es el núcleo de la operación **list_directory** y la fase previa obligatoria en procesos de **download_folder**, **copy_folder**, **move_folder** y **compress_directory**. En estos casos, el cliente requiere indexar el contenido antes de iniciar cualquier transferencia de datos.

- **Borrado jerárquico:** Debido a que SMB2 no permite la eliminación directa de directorios con contenido, el comando FIND es esencial para listar los elementos hijos, proceder a su borrado individual y, finalmente, eliminar el nodo raíz una vez vacío.

A pesar de que este comando aparece de forma recurrente en operaciones sobre archivos individuales —debido a que el explorador de Windows refresca el listado para evitar colisiones de nombres o actualizar la interfaz— esta metodología establece que el **inicio de la operación semántica reside en el comando SMB2_CREATE**. Esta decisión permite centrar el perfilado de usuario en la interacción directa con el recurso, descartando el tráfico de fondo generado por el sistema operativo.

Modelo de análisis sobre carpetas vacías y replicación lógica

Para garantizar la precisión de la Máquina de Estados (FSM), las operaciones de directorio (borrado, copia y descarga) se han modelado sobre **carpetas sin contenido**. Este enfoque permite observar de forma aislada el comportamiento del protocolo sobre el contenedor, evitando que la transferencia masiva de archivos internos distorsione la secuencia de control capturada.

Bajo este modelo, el comando FIND asume el rol de validador de estado:

- **Caso base (Carpeta vacía):** La respuesta **STATUS_NO_MORE_FILES** permite a la FSM cerrar la operación de forma atómica, sin necesidad de gestionar elementos dependientes.
- **Caso recursivo (Carpeta con contenido):** El listado de archivos en la respuesta indica que la operación principal debe permanecer en espera mientras se ejecutan los ciclos independientes de **CREATE**, **READ/WRITE** y **CLOSE** para cada archivo hijo.

En conclusión, una operación recursiva se define técnicamente como una **iteración de operaciones de fichero** precedida por un comando FIND. Este diseño asegura la **validez del modelo ante cualquier volumen de datos**, permitiendo que la lógica aplicada a una carpeta vacía se repita de forma sistemática para cada archivo detectado, vinculándolos bajo el mismo identificador (**FileID**) y de sesión (**TreeID**).

1.3 Gestión de metadatos y filtrado de tráfico auxiliar

En la observación de trazas reales de SMB2, se detecta un volumen significativo de tráfico que no corresponde a la ejecución de la acción semántica del usuario, sino a la gestión interna del sistema operativo cliente. Para obtener perfiles de actividad limpios, se ha procedido a identificar y filtrar este tráfico auxiliar.

El comando SMB2_GET_INFO y la recolección de metadatos

El comando **SMB2_GET_INFO** (0x10) se utiliza para consultar atributos específicos de un archivo o del propio sistema de archivos del servidor. En las capturas realizadas, este comando aparece de forma masiva acompañando a casi todas las operaciones. Su presencia se debe a tres funciones principales del explorador de archivos:

- **Visualización:** Obtención de atributos para mostrar iconos, fechas de modificación y tamaños de archivo.
- **Gestión de Cuotas:** Consulta del espacio disponible en el volumen de red.

- **Seguridad:** Verificación de las Listas de Control de Acceso (ACL) para determinar qué acciones puede mostrar la interfaz al usuario (borrar, editar, etc.).

Dado que estas consultas no alteran el estado de los datos ni representan una intención operativa del usuario sobre el fichero, **se han filtrado de la fase de análisis** para evitar que la densidad de mensajes oculte la lógica de la operación principal.

Ciclos de apertura y cierre efímeros (Create/Close)

Un fenómeno recurrente en las capturas es la aparición de secuencias **CREATE** → **CLOSE** o **CREATE** → **GET_INFO** → **CLOSE** inmediatamente antes o después de la operación clave (como un **READ** o un **WRITE**). Estos ciclos los vamos a definir como **aperturas efímeras**.

El sistema operativo realiza estas aperturas para inspeccionar el archivo sin intención de operar con él. Por ejemplo, al pasar el ratón sobre un archivo o al abrir una carpeta, Windows abre y cierra archivos rápidamente para leer sus cabeceras.

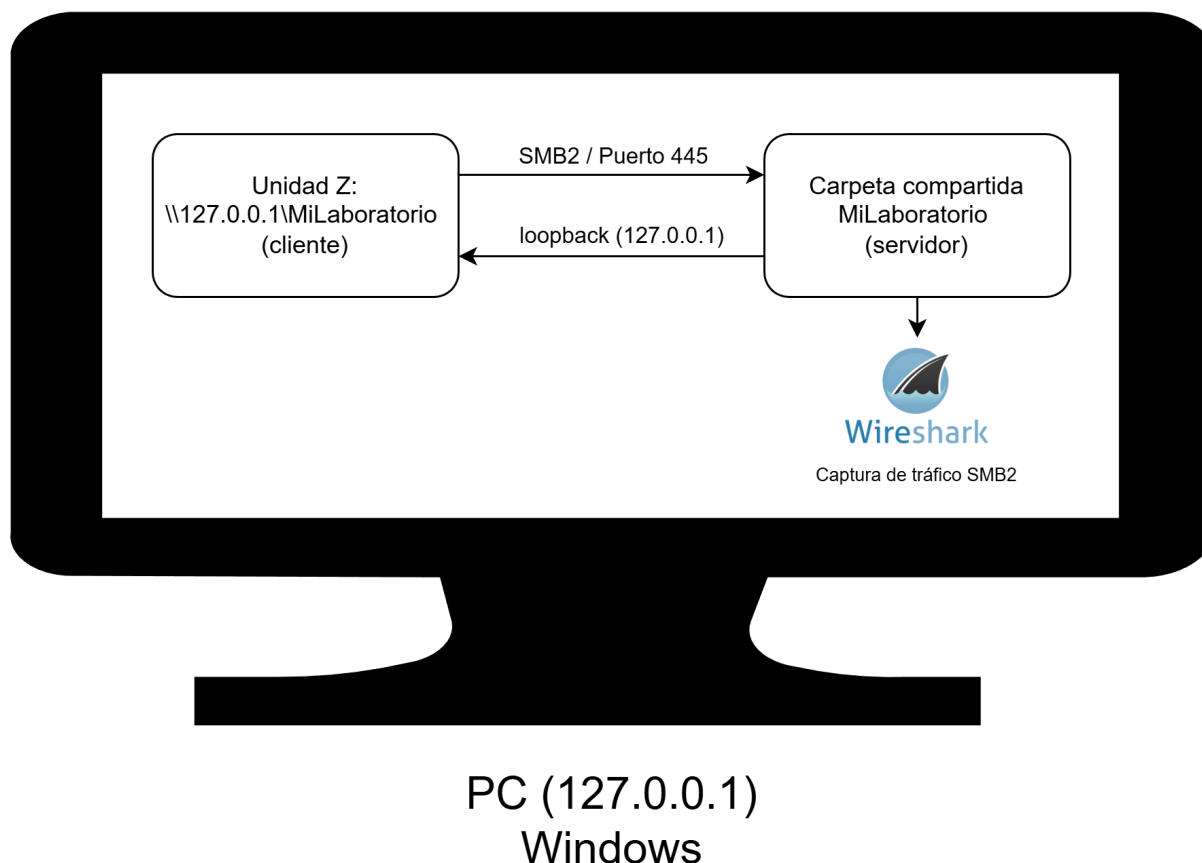
- **Criterio de filtrado:** En esta metodología, se descartan todos los ciclos **CREATE** / **CLOSE** que no contengan operaciones de datos (**READ**, **WRITE**) o de control de estructura (**QUERY_DIRECTORY**, **SET_INFO** para renombrado, etc.).
- **Justificación:** Mantener estos ciclos en la Máquina de Estados (FSM) generaría una sobrecarga de eventos que no aportan valor analítico y descriptivo a la actividad real del usuario en la cabina de datos.

El proceso de apertura y consulta de recursos sigue el procedimiento descrito en [1, 3.2.4.3] y [1, 3.2.4.8], donde se establece que el cliente debe inicializar una petición **SMB2 CREATE** seguida de un comando **SMB2 QUERY_INFO** para obtener el **FileInfoClass** solicitado por la aplicación (por ejemplo, al visualizar metadatos en el Explorador de Windows), con parámetros específicos como **DesiredAccess**, **ShareAccess** y **CreateOptions**. Una vez obtenidos los atributos, el cliente invoca el procedimiento de **cierre inmediato** [1, 3.2.4.5] mediante un comando **SMB2 CLOSE**. La observación de estos ciclos múltiples sin transferencia de datos (**READ** / **WRITE**) permite su clasificación como tráfico auxiliar y justifica su posterior filtrado para limpiar el perfil de actividad real del usuario.

1.2 Evolución del experimento

1.2.1 Entorno local (Propuesta inicial)

Para solucionar esto y poder ver el protocolo SMB2 de verdad, decidimos cambiar el escenario. En lugar de usar Dropbox, creamos una **unidad de red compartida directamente en Windows [5]**.



Este cambio ha sido clave por varios motivos:

- **Vemos la realidad del protocolo:** Ahora, al ser una unidad de red pura, Windows no tiene más remedio que usar el puerto 445 y el protocolo SMB2 para todo.
- **Aparecen los datos:** En este nuevo entorno, por fin podemos ver los paquetes **READ (lectura)** y **WRITE (escritura)**. Ahora sí vemos cómo el archivo se trocea en paquetes y viaja de un punto a otro, que es exactamente lo que queremos documentar en este trabajo.

Conclusión

Este ajuste en el experimento nos ha permitido pasar de una captura "contaminada" por el software de Dropbox a una captura limpia y profesional. Ahora los resultados coinciden con la teoría: cada vez que descargamos un archivo, vemos la secuencia completa de apertura, la ráfaga de paquetes con los datos reales y el cierre final.

Montaje unidad virtual

New-PSDrive -Name "Z" -PSProvider FileSystem -Root "\\127.0.0.1\MiLaboratorio" -Persist

1.2.2 Evolución del Escenario Experimental: Hacia un Entorno Distribuido Real

El diseño del experimento ha evolucionado desde pruebas iniciales en entornos de bucle local hasta una arquitectura de red pura entre dos equipos físicos. Este proceso ha sido fundamental para aislar el comportamiento de SMB2 de cualquier proceso de optimización externa del sistema operativo.

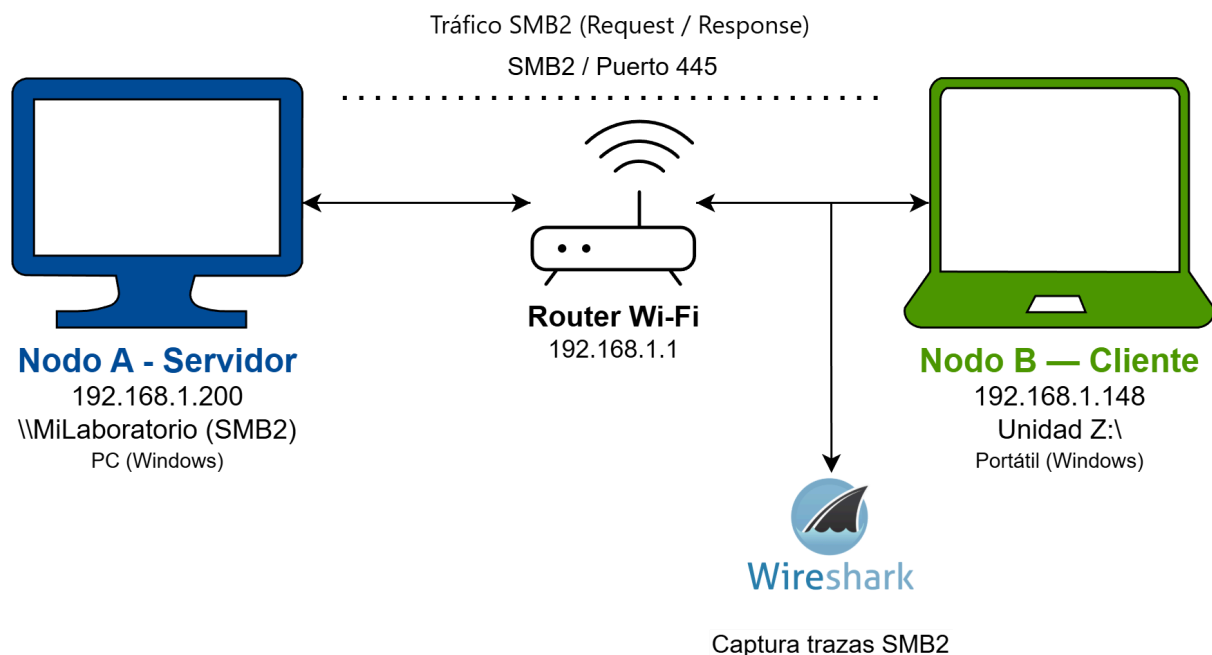
A. El problema de la optimización: La limitación del Bucle Local

Inicialmente, se realizaron pruebas utilizando una unidad de red mapeada en bucle local (127.0.0.1). Este escenario presentó limitaciones críticas para la auditoría de tráfico:

- **Gestión por Memoria Caché:** En el bucle local, el sistema operativo detecta que el origen y el destino de los datos residen en la misma máquina física. Como consecuencia, gestiona la transferencia mediante la memoria caché del sistema.
- **Invisibilidad de Datos:** Esta optimización provocaba que las ráfagas de datos no llegaran a atravesar la pila de red física, haciendo que los paquetes **READ** y **WRITE** fueran invisibles para el motor de captura de Wireshark. El resultado eran trazas compuestas casi exclusivamente por mensajes de control, sin representación del flujo de bytes real.

B. Configuración del Entorno con dos Equipos (Análisis vía Wi-Fi)

Para obtener capturas "limpias" y profesionales, se decidió separar físicamente los nodos: un **PC Servidor (Nodo A)** y un **PC Cliente/Portátil (Nodo B)**.



1. Estrategia de Acceso Simplificado

Para garantizar que el protocolo SMB2 mostrará su comportamiento de transferencia sin bloqueos por políticas de dominio o credenciales locales, se optó por un entorno de **Acceso Abierto Controlado**:

- **Servidor (Nodo A):** Se desactivó el "Uso compartido con protección por contraseña" en el Centro de redes y recursos compartidos.
- **Creación del Recurso:** Se ejecutó el comando:
`New-SmbShare -Name "MiLaboratorio" -Path "C:\PruebaSMB" -FullAccess "Todos"`
Esto alineó los permisos de red con los permisos NTFS, eliminando cualquier "Access Denied" en la capa de negociación.

2. Validación y Comportamiento en el Aire (Wi-Fi)

En el cliente (Nodo B), la conexión se realizó de forma directa: `New-PSDrive -Name "Z" -PSProvider FileSystem -Root "\\192.168.1.200\MiLaboratorio" -Persist [12]`

Al no haber credenciales de por medio, el análisis en Wireshark revela de forma muy pura la **Máquina de Estados de SMB2**:

- Al realizar las operaciones desde el portátil conectado por Wi-Fi, el sistema perdió cualquier capacidad de optimización local. Los archivos se vieron obligados a fragmentarse en paquetes SMB2 reales para viajar por el aire.

C. Conclusión del Cambio de Escenario

Este ajuste metodológico permitió pasar de una captura "contaminada" o invisible a una **captura real**. Al operar sobre una red física, el tráfico capturado en el portátil coincide finalmente con la teoría: cada descarga forzada desde la unidad **Z:** genera la secuencia completa de apertura de sesión, ráfagas de paquetes de datos (**READ Response**) y el cierre final.

1.3 Analisis Operaciones de Usuario

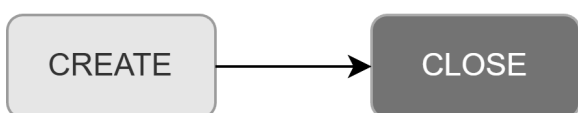
Para construir el motor de clasificación del analizador, se procedió a monitorear y modelar de forma aislada las acciones más frecuentes en la gestión de archivos dentro del entorno experimental. Con el fin de agilizar la lectura de esta memoria y priorizar el enfoque metodológico, **el catálogo exhaustivo de las operaciones, su descripción funcional y el desglose técnico de sus secuencias lógicas de comandos (flujos atómicos) y trazas de red se han unificado en el Anexo D.**

En este apartado se exponen, por lo tanto, los principios de diseño fundamentales y los diagramas conceptuales de secuencia que permiten abstraer dicho comportamiento para alimentar la Máquina de Estados (FSM) del analizador.

1.3.1 Create operations

1.3.1.1 Create folder operation

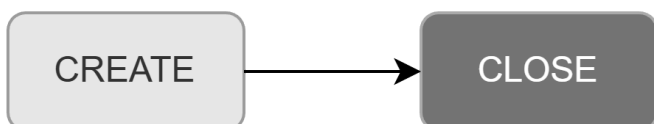
La operación **Create Folder** (ejecutada mediante el comando **mkdir**) consta de una solicitud de apertura de objeto donde el cliente especifica que el destino debe ser tratado como un contenedor jerárquico. Esta operación es una de las más sencillas, ya que **se define por el estado de activación de un solo bit de control** en la cabecera de la petición.



Comando: **mkdir Z:\CarpetaVacía**

1.3.1.2 Create file operation

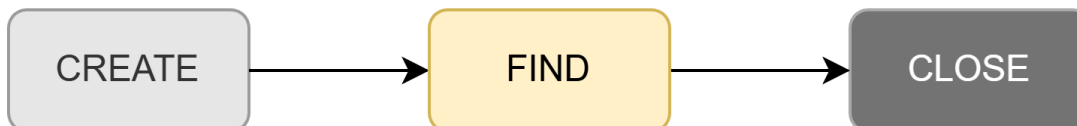
La operación **Create File** consta de una secuencia idéntica en estructura, pero con una instrucción lógica inversa en su máscara de bits. Esta operación es, asimismo, una de las más sencillas, ya que **reutiliza la misma primitiva de comunicación** variando únicamente la señalización del objeto para indicar que se trata de un elemento de datos.



1.3.2 Data Movement Operations:

1.3.2.1 Upload folder operation [Anexo C - Captura 1]

En el protocolo SMB2, la acción de subir una carpeta se diferencia según la naturaleza de su contenido. Si se trata de una Carpeta Vacía, la operación se considera atómica y consiste únicamente en la creación de metadatos en el servidor a través de la secuencia CREATE (con paridad impar en CreateOptions) y CLOSE, sin que exista transferencia de datos real (WRITE).

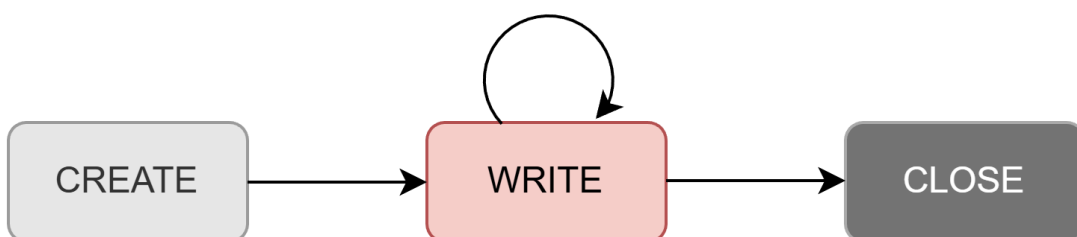


Comando: **xcopy "C:\Users\MaRio\Downloads\CarpetaVacía2" "Z:\CarpetaVacía2" /E /IS**

Sin embargo, cuando se sube una **Carpeta con Contenido**, el protocolo no ejecuta un comando único de subida, sino que realiza una operación **concatenada** o **recursiva**. En este escenario, el sistema descompone la acción abriendo primero la carpeta raíz y, antes de cerrarla, inicia múltiples flujos secundarios para cada archivo y subcarpeta interior. Para el análisis de comportamiento, esto significa que una subida compleja es en realidad la suma de varias operaciones individuales que ocurren de forma simultánea o sucesiva bajo un mismo contexto temporal, vinculadas por el rastro de sus respectivos identificadores de archivo.

1.3.2.2 Upload file operation: [Anexo C - Captura 2]

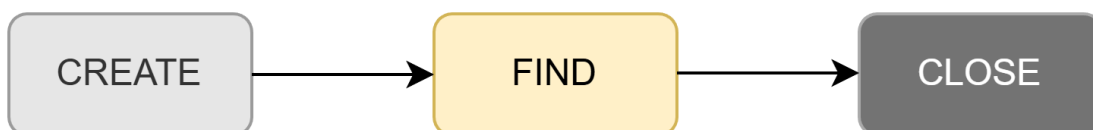
Consiste en la transferencia física de la carga útil de un fichero regular hacia el servidor de almacenamiento. Se diferencia de la subida de directorios en que el analizador busca la presencia explícita de comandos de escritura. El tamaño total del objeto transferido se calcula de forma automatizada mediante la agregación acumulativa de las longitudes de los bloques de red.



Comando: **copy "C:\Users\MaRio\Downloads\archivo.txt" "Z:\"**

1.3.2.3 Download folder operation [Anexo C - Captura 3]

En el protocolo SMB2, la acción de descargar una carpeta se gestiona como una lectura de estructura y datos desde el servidor. Si se trata de una **Carpeta Vacía**, la operación consiste únicamente en la apertura y consulta de metadatos mediante la secuencia **CREATE** (con paridad impar en **CreateOptions**), un comando **FIND** (para verificar que no hay contenido) y su posterior **CLOSE**, sin que exista movimiento de bits de datos (**READ**).



Comando: **copy "Z:\CarpetaVacía\" "C:\Downloads\"**

Sin embargo, cuando se descarga una **Carpeta con Contenido**, el protocolo realiza una **operación concatenada o recursiva**. En este escenario, el sistema primero explora la carpeta raíz mediante el comando **FIND** para obtener el listado de archivos. Una vez obtenida esta información, inicia múltiples flujos secundarios de lectura mediante el comando **READ (0x08)** para cada archivo interior. Para el análisis de comportamiento, esto significa que la descarga es la suma de una fase de navegación (listado) y una fase de transferencia, donde los datos fluyen del servidor al cliente, quedando vinculadas todas las acciones por el contexto del **Tree ID** y los respectivos identificadores de archivo.

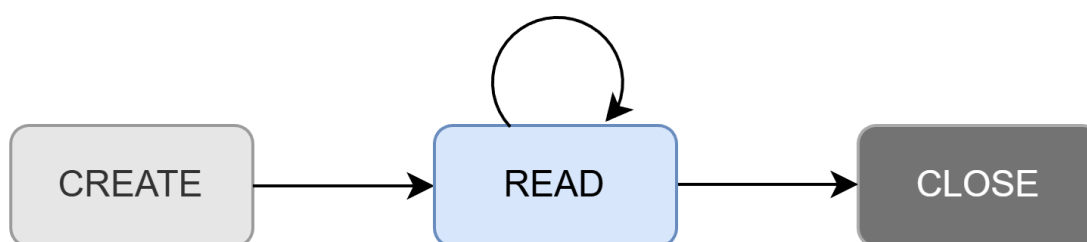
1.3.2.4 Download file operation [Anexo C - Captura 4]

Esta operación describe la transferencia de datos desde el servidor (almacenamiento en la nube) al cliente local.

La identificación de los mensajes de lectura (**READ 0x08**) supuso el mayor desafío técnico del laboratorio. Durante las fases iniciales, el uso de una única máquina para simular cliente y servidor impidió la captura de estos paquetes, lo cual podría deberse a varias causas técnicas concurrentes recogidas en el **Anexo D**.

Implementación y resolución: Para mitigar estas incertidumbres, se optó por separar físicamente los nodos, realizando la operación desde el equipo portátil hacia el equipo servidor. Al ejecutar la copia de un archivo desde la unidad de red (Z:) hacia un directorio local a través de una conexión Wi-Fi, se forzó al protocolo a adaptarse a las limitaciones de una red física real.

Este cambio de estrategia permitió confirmar que el comando **READ** es la unidad básica de transferencia desde el servidor al cliente, y que su correcta monitorización en este proyecto requería un entorno distribuido para compensar la velocidad y las optimizaciones de las conexiones locales.

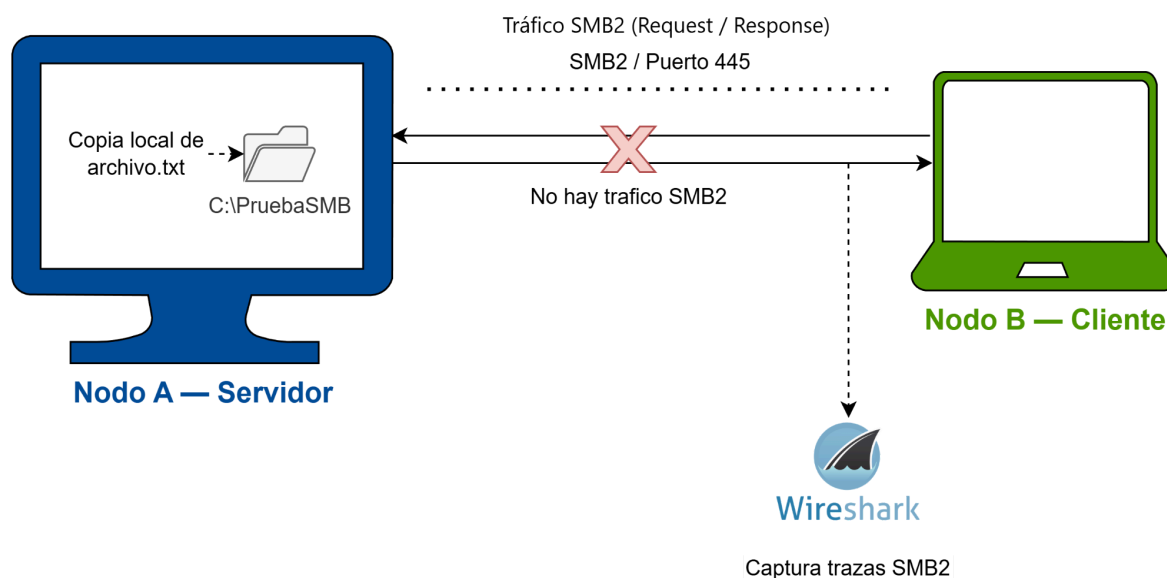


Comando: **copy "Z:\ejemplo.txt" "C:\carpetaEjemplo"**

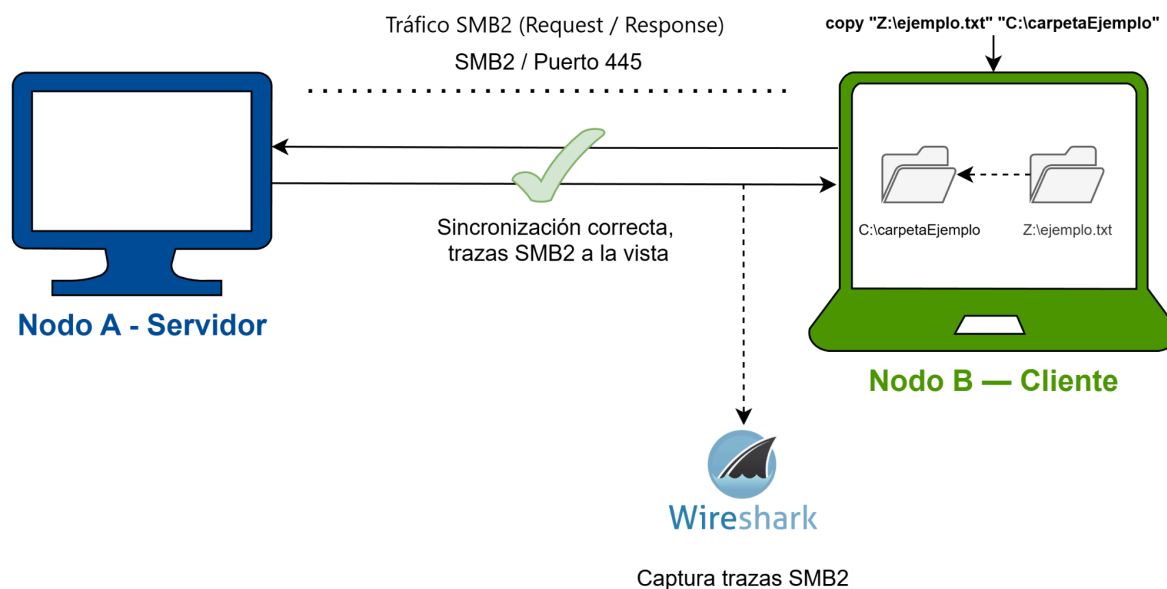
Simulación de la operación de descarga (Download)

La propuesta inicial consistía en subir un archivo directamente al servidor (PC A), esperando que el cliente (PC B) lo detectará y generará tráfico capturable. Sin embargo, SMB2 no implementa sincronización activa: al no existir mecanismos *push*, el cliente no genera ningún paquete hasta que él

mismo inicia una solicitud. El resultado fue que Wireshark no registraba actividad alguna.



Para resolverlo, se redefinió el método ejecutando desde el cliente el comando **copy "Z:\ejemplo.txt" "C:\carpetaEjemplo"**. Al operar sobre la unidad de red mapeada Z:, el sistema operativo se ve obligado a solicitar los datos al servidor a través del puerto 445, generando la secuencia completa **CREATE** → **READ** → **CLOSE** que el analizador sí puede registrar. El flujo de bits es técnicamente equivalente a una descarga real en un entorno corporativo. El detalle completo de este proceso se recoge en el Anexo D.

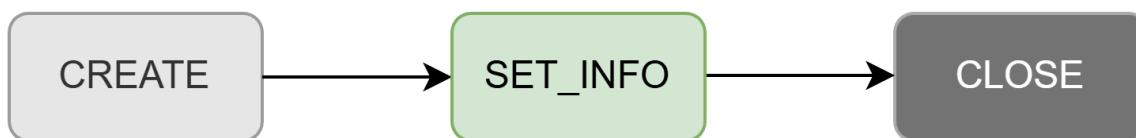


1.3.3 Metadata Operations and Organization:

1.3.3.1 Rename folder operation [Anexo C - Captura 5]

La operación de renombrado en SMB2 consiste en la modificación de los metadatos de un objeto (archivo o carpeta) para alterar su identificador nominal sin mover físicamente los datos en el disco

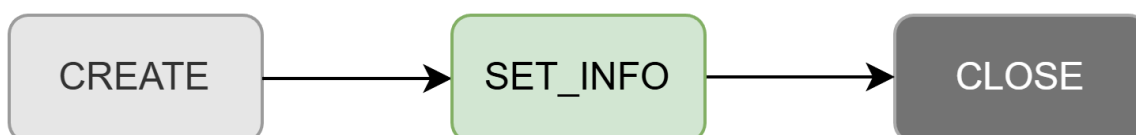
duro. A diferencia de una operación de lectura, donde se transfiere contenido, el renombrado es una operación de control que comunica al servidor un cambio de estado en el sistema de archivos remoto.



Comando: **ren "Z:\prueba\" pruebaNueva**

1.3.3.2 Rename file operation [Anexo C - Captura 6]

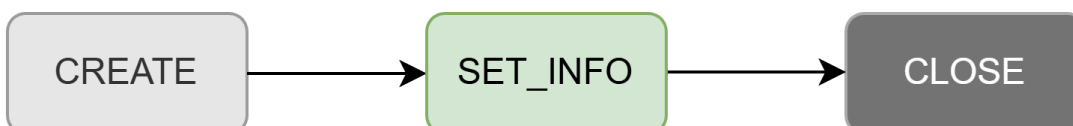
Sigue una secuencia idéntica al renombrado de directorios, al tratarse de una modificación exclusiva de metadatos nominales que no altera los bloques físicos del recurso. La distinción clave para el analizador radica en la fase de apertura (**SMB2_CREATE**), donde los flags de control confirman que la transacción se ejecuta sobre un elemento de datos regular y no sobre un contenedor jerárquico.



Comando: **ren "Z:\prueba.txt" prueba1.txt**

1.3.3.3 Move folder operation: [Anexo C - Captura 7]

Esta operación sirve para cambiar una carpeta completa de sitio dentro del mismo recurso compartido. En el protocolo SMB2, mover una carpeta no significa volver a subir los datos, sino actualizar la estructura de directorios en el servidor. Aunque la secuencia de comandos es igual que la de un renombrado, la clave aquí es que dentro del comando **SET_INFO** (en la clase **FileRenameInformation**), el campo **FileName** cambia y pasa a llevar la ruta completa de destino. Así es como el servidor sabe que tiene que mover todo el árbol de carpetas a la nueva ubicación.

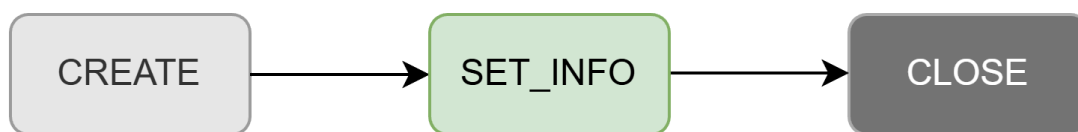


Comando: **Move-Item "Z:\CarpetaOrigen" "Z:\NuevaRuta\"**

1.3.3.4 Move file operation:

Consiste en mover un único archivo de una carpeta a otra dentro del propio servidor, cambiando sus punteros lógicos sin necesidad de mover los bloques físicos de datos en el disco. La diferencia principal con el movimiento de carpetas es que esta acción es atómica (afecta a un solo archivo independiente). El campo **FileName** del paquete **SET_INFO** indica la ruta de la carpeta de destino junto con el nombre del fichero. Para distinguirlo del movimiento de carpetas en el analizador, nos

fijamos en la petición **CREATE** inicial, comprobando que el bit de directorio (**FILE_DIRECTORY_FILE**) está en cero.

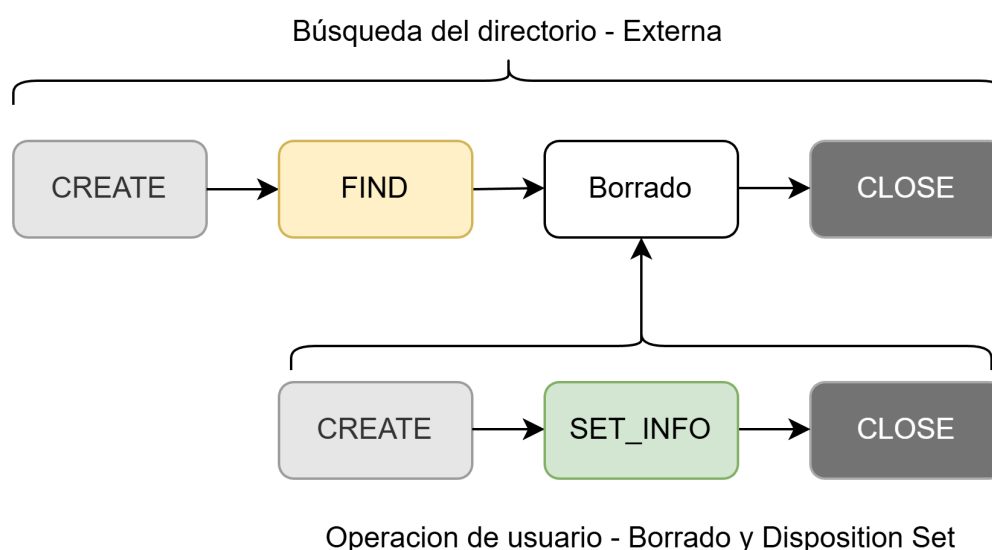


Comando: **Move-Item "Z:\documento.pdf" "Z:\CarpetaDestino\"**

1.3.4 Deletion and Query Operations:

1.3.4.1 Delete folder operation [Anexo C - Captura 8] [Anexo C - Captura 9]

Esta operación describe la eliminación de un directorio en el sistema de archivos del servidor. En el protocolo SMB2, el borrado de una carpeta se gestiona como una instrucción de *disposición*. Antes de la operación de usuario es esencial el comando find, ya que tiene que además de buscar si existe la carpeta a borrar, si dentro de ella existe algo de lo que tenga que ocuparse para hacer operaciones recursivas de borrado.



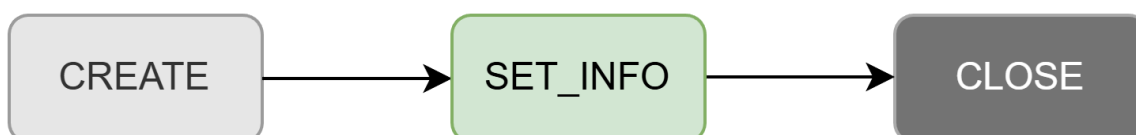
Comando: **rd Z:\Prueba**

Borrado de Carpeta con Contenido: Cuando se ejecuta una eliminación recursiva, el protocolo no envía un único comando de borrado total. En su lugar, el cliente activa una **secuencia iterativa**: primero realiza un **FIND** para listar los elementos, ejecuta una operación de borrado (**SET_INFO** + **CLOSE**) por cada archivo y subcarpeta interna, y solo cuando el directorio raíz queda vacío, procesa su eliminación definitiva.

1.3.4.2 Delete file operation: [Anexo C - Captura 10]

El borrado de un archivo individual sigue la misma lógica de *disposición* que la carpeta, comunicando al servidor que el objeto ya no es necesario.

El comando de eliminación surte efecto al cerrar el archivo. Por lo tanto, primero se debe abrir el recurso. Luego, se le asigna la etiqueta **FileDispositionInformation**. Finalmente, la eliminación se produce al cerrar el archivo.

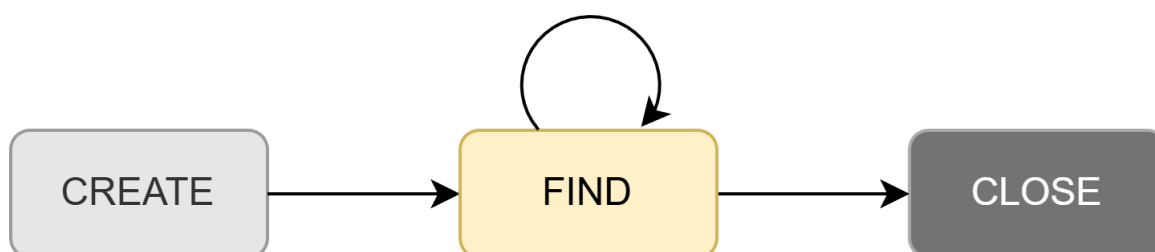


Comando: **rd Z:\prueba.txt**

Mecanismo: El cliente marca el archivo para su eliminación. Es importante notar que, si otro proceso tiene el archivo abierto, el servidor esperará a que se cierren todos los *handles* antes de borrarlo definitivamente.

1.3.4.3 List directory operation: [Anexo C - Captura 11]

El comando identificado por Wireshark como FIND corresponde técnicamente al comando 0x0E (SMB2 QUERY_DIRECTORY). Se utiliza para obtener un listado de los objetos contenidos en un directorio mediante el uso de patrones de búsqueda



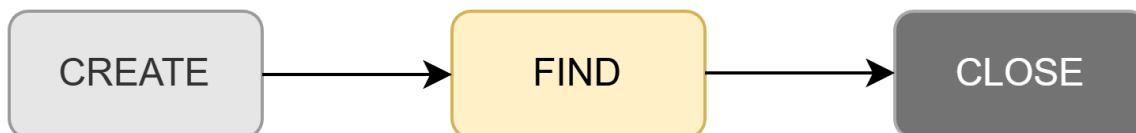
Comando: **dir Z:**

1.3.5 Copy and Compression Operations:

1.3.5.1 Copy folder operation:

En el protocolo SMB2, la acción de copiar una carpeta se gestiona de forma distinta dependiendo de si el directorio contiene objetos o no.

Copia de Carpeta Vacía: Como venimos diciendo en muchas de las anteriores operaciones, se considera una operación puramente administrativa o de metadatos. El cliente solicita al servidor la reserva de un nuevo espacio en el sistema de archivos mediante una secuencia de apertura y cierre, sin que exista transferencia de carga útil.



Comando: **copy "C:\Ruta\CarpetaVacía" "Z:\"** (Subida)

copy "Z:\" "C:\Ruta\CarpetaVacía" (Descarga)

En ambos sentidos, la traza es **indistinguible**, ya que el servidor solo procesa la creación del contenedor y no hay transferencia de carga útil que delate la dirección del flujo.

Copia de Carpeta con Contenido: El protocolo no ejecuta un comando global de "duplicación", sino que realiza una **operación recursiva y concatenada**. El sistema inicia abriendo la carpeta raíz y, antes de cerrarla, dispara flujos secundarios para cada elemento interno. Para el análisis en Wireshark, esto se traduce en una sucesión masiva de comandos donde se intercalan peticiones CREATE para nuevas subcarpetas con ciclos de READ y WRITE para los archivos, todos vinculados cronológicamente pero con diferentes identificadores de archivo (*File ID*).

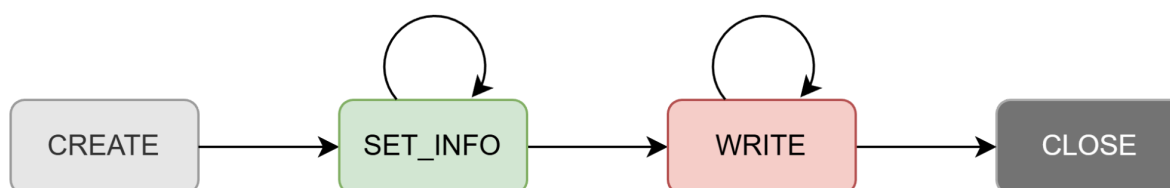
1.3.5.2 Copy file operation:

La copia de un archivo individual es la operación fundamental para observar cómo se transporta la carga útil a través de la red. A diferencia de un movimiento, donde solo se cambian etiquetas de sitio, la copia obliga a que los bits del archivo viajen físicamente de un disco a otro.

Para documentar este proceso de forma real y evitar que el sistema operativo use "atajos" de memoria, se han realizado pruebas en las dos direcciones posibles:

Flujo de Subida (Copia de local a red) [Anexo C - Captura 12]

Cuando copiamos un archivo desde el disco local C: hacia la unidad de red Z:, el portátil actúa como emisor y el PC servidor como receptor.

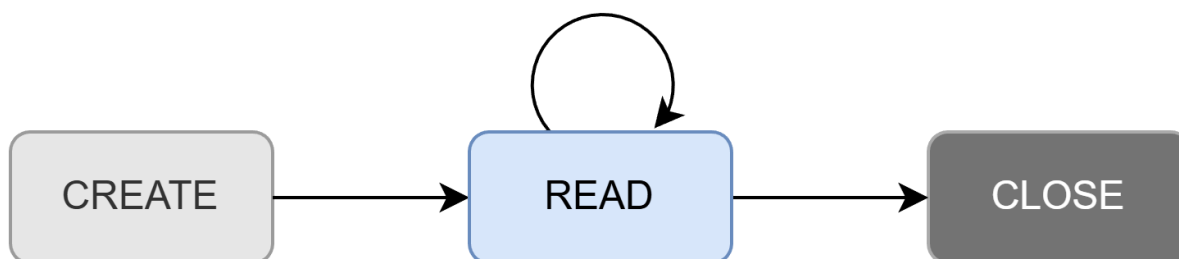


Comando: **copy "C:\Ruta\archivo.txt" "Z:\"**

En el flujo de subida, la presencia de **SMB2 SET_INFO** es fundamental para replicar la integridad del archivo original en el servidor, ya que permite gestionar los metadatos que el comando **WRITE** no transporta. Estos paquetes se encargan de reservar el espacio físico en el disco remoto mediante **FileEndOfFileInfo** (evitando la fragmentación), sincronizar atributos temporales como la fecha de creación y modificación a través de **FileBasicInfo**. Mientras que los paquetes de escritura transfieren

la carga útil de bits, los de información aseguran que el archivo resultante en la unidad **Z:** mantenga la misma identidad, estructura y privilegios que el original en **C:\PruebaSMB**.

Flujo de Descarga (Copia de red a local) [Anexo C - Captura 13]



Comando: **copy "Z:\archivo.txt" "C:\Ruta\"**

El análisis en ambos sentidos permite confirmar una regla básica del protocolo: aunque la estructura de apertura (**CREATE**) y cierre (**CLOSE**) es casi idéntica, el tipo de paquete que transporta los datos es distinto. En la subida predominan las peticiones de escritura (**WRITE**), mientras que en la descarga el tráfico principal son las respuestas de lectura (**READ Response**).

Esta distinción es clave para identificar en una captura de red qué equipo está enviando la información y asegurar que el archivo ha llegado íntegro a su destino.

1.3.5.3 Compress folder operation:

Al intentar comprimir una **Carpeta Vacía** mediante el comando Compress-Archive, se observa que el archivo .zip puede no llegar a generarse o no producir tráfico de datos relevante.

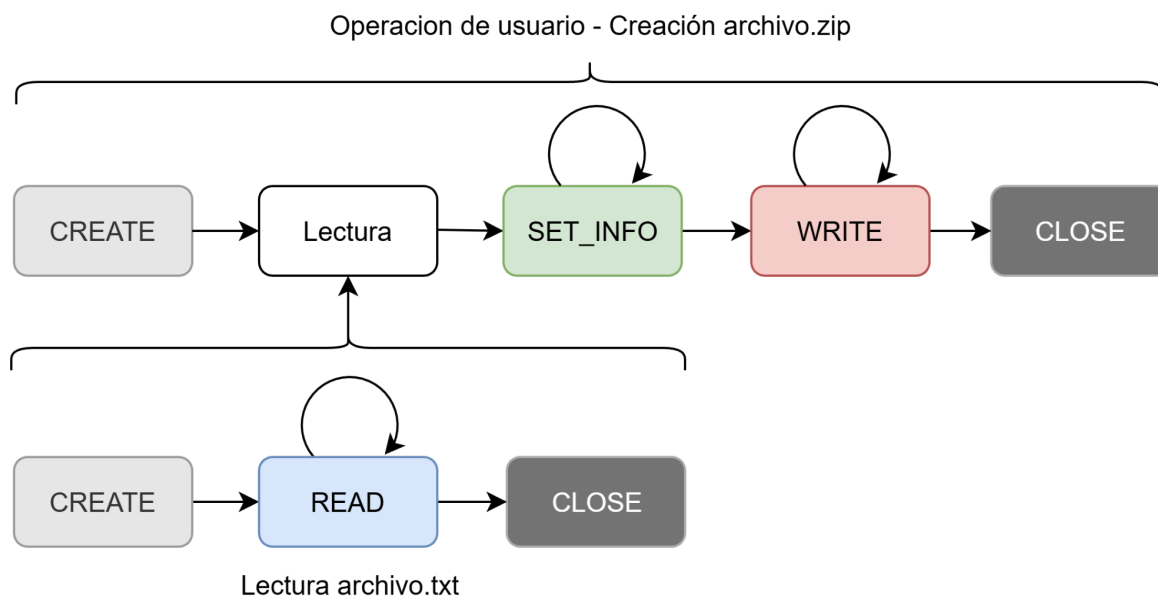
Explicación técnica: El motor de compresión de Windows requiere un flujo de entrada para inicializar la estructura del archivo comprimido. Si el comando no detecta archivos (metadatos de contenido), la operación puede finalizar tras un **CREATE** y **CLOSE** inmediato sin llegar a ejecutar el **WRITE**, ya que no hay una carga útil que justifique la creación del contenedor en el servidor.

La compresión de una carpeta completa añade una capa de complejidad al protocolo, ya que requiere la gestión de múltiples objetos de forma secuencial.

Flujo con Contenido [Anexo C - Captura 14]: Si la carpeta contiene archivos, el protocolo entra en un ciclo iterativo. Por cada elemento, se repite la secuencia de apertura, lectura y posterior escritura hacia el archivo **.zip** único. Esto genera una concatenación de operaciones en la captura de red proporcional al número de archivos internos.

1.3.5.4 Compress file operation: [Anexo C - Captura 15]

La operación de compresión permite observar cómo el protocolo SMB2 interactúa con procesos de transformación de datos en tiempo real. A diferencia de una copia simple, aquí el equipo cliente debe procesar la información antes de enviarla al destino. Para el análisis, se ha utilizado el motor nativo de PowerShell mediante el comando **Compress-Archive**.



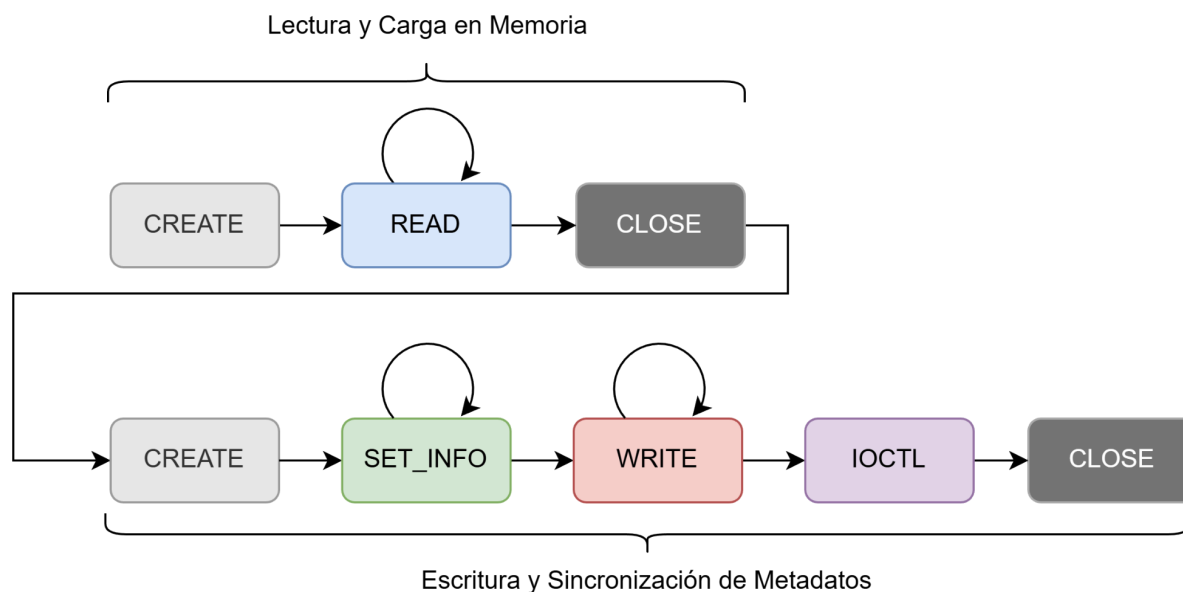
Como se ilustra en el diagrama de estados, la compresión no es una acción atómica, sino la combinación de dos operaciones esenciales entrelazadas: la **lectura del origen** y la **creación del contenedor destino**.

Comando: **Compress-Archive -Path "Z:\prueba.txt" -DestinationPath "Z:\prueba.zip"**

1.3.6 Modification Operation:

1.3.6.1 Modify file operation: [Anexo C - Captura 16]

La modificación de un archivo es una operación crítica que permite observar la capacidad de SMB2 para gestionar la integridad de los datos y la actualización de metadatos en tiempo real. A diferencia de una copia, la modificación implica que el cliente interactúa con un archivo ya existente, alterando su contenido y sus propiedades sin crear un objeto nuevo.



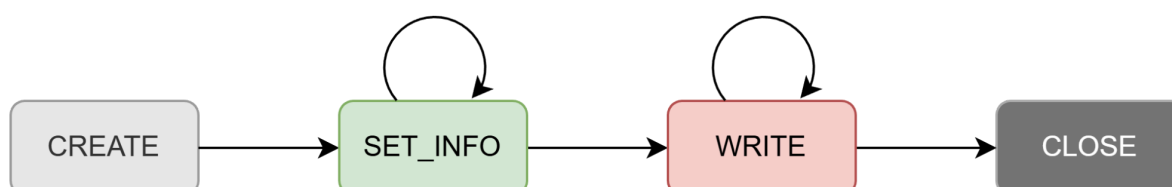
Comando: **notepad Z:\prueba.txt**

El estudio de esta operación revela que SMB2 es un protocolo **interactivo** y no solo de transporte. El uso constante de comandos **SET_INFO** e **IOCTL** durante la modificación demuestra que el cliente asume el control total de la estructura del archivo en el servidor.

Esta distinción es fundamental para el proyecto: la modificación de un archivo genera un tráfico de control mucho más denso que una copia simple, ya que debe asegurar que los metadatos del sistema de archivos y los datos reales estén perfectamente sincronizados en todo momento, evitando la corrupción de datos en caso de una desconexión accidental.

Optimización del método de modificación: Notepad vs “comando atomico” [Anexo C - Captura 17]

Durante la fase de experimentación, se evaluó el uso del editor de texto **Notepad** para la modificación de archivos. El problema viene de que la apertura de una interfaz gráfica genera un volumen excesivo de tráfico redundante. Notepad realiza múltiples peticiones de metadatos, carga de recursos del sistema y comprobaciones de estado de ventana que no aportan valor al análisis del protocolo SMB2 y dificultan la identificación del flujo de datos real.



Para obtener una captura limpia y centrada exclusivamente en la escritura de datos, se optó por la ejecución del comando en consola: **cmd /c "copy con Z:\prueba.txt"**

La transición hacia el comando **copy con** permitió aislar el comportamiento operativo del protocolo SMB2. Mientras que las herramientas gráficas introducen latencias y tráfico de control asíncrono, el método de consola revela una secuencia atómica y lineal de los estados de modificación.

Este análisis confirma que la modificación de un archivo remoto requiere gestionar la estructura del recurso —asignando espacio mediante **SET_INFO**— antes de proceder a la transferencia de datos con **WRITE**. El valor de esta metodología está en capturar el momento exacto de persistencia tras la señal de cierre (**CLOSE**), lo que produce una visión reproducible del comportamiento del protocolo en tareas de edición.

4. DESARROLLO Y EXPERIMENTACIÓN

1. Introduccion

1.1 Objetivo

El objetivo de este análisis es, a partir de una traza de red capturada con Wireshark, identificar automáticamente las operaciones de usuario que se han realizado: subir archivos, descargarlos, borrarlos, renombrarlos, crear carpetas, etc.

El código fuente del analizador se encuentra en [src/analizador_smb2_v4.py](#). Las versiones anteriores (v1, v2, v3) se mantienen como referencia historica en [src/analizador_smb2.py](#), [src/analizador_smb2_v2.py](#) y [src/analizador_smb2_v3.py](#).

1.2 Flujo general del analisis

El proceso completo sigue estos pasos:

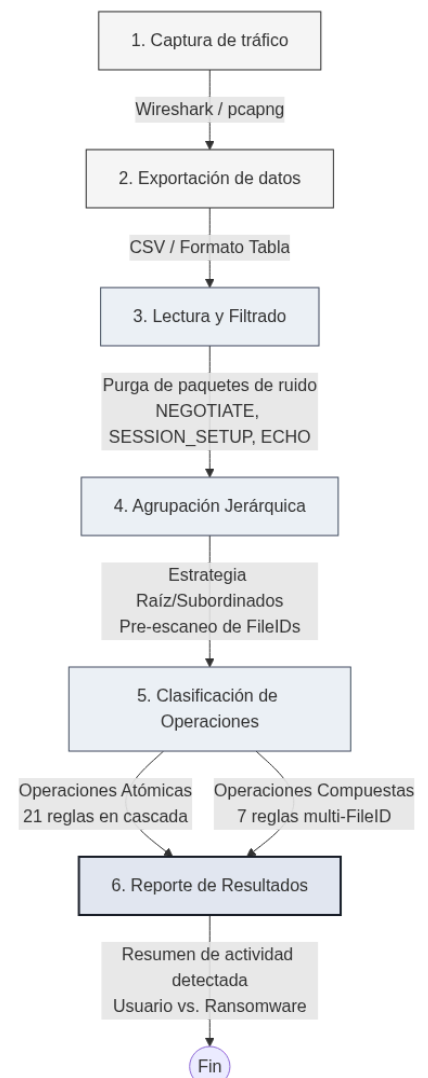
1. **Captura de trafico** con Wireshark en formato pcapng
2. **Exportacion** a CSV con columnas SMB2, o a formato tabla desde Wireshark
3. **Lectura del archivo** y filtrado de paquetes de ruido (NEGOTIATE, SESSION_SETUP, etc.)
4. **Agrupacion jerarquica** usando la estrategia raiz/subordinados con pre-escaneo de FileIDs
5. **Clasificacion** de cada operacion: las atomicas con 21 reglas en cascada, las compuestas con 7 reglas multi-FileID
6. **Generacion del reporte** con el resumen de operaciones detectadas

1.3 Estructura del archivo CSV

Wireshark permite exportar paquetes capturados a formato CSV con columnas personalizadas. Las columnas relevantes para este análisis, junto con su descripción completa, se recogen en el **Anexo E**. Los campos esenciales son el timestamp relativo del paquete (**frame.time_relative**), el comando SMB2 (**smb2.cmd**), el código de estado (**smb2.nt_status**), el **TreeID**, el **FileID**, las opciones de creación (**smb2.create_options**), la clase de información para comandos **SET_INFO** (**smb2.set_info.info_class**) y la ruta del archivo (**smb2.filename**).

Ademas del formato CSV, el analizador v4 soporta un **formato TXT espacio-separado** (generado por una herramienta de captura diferente), usado para trazas de gran volumen como **salidaSemana.txt**. La deteccion del formato es automatica.

Existen comandos SMB2 que son gestionados automaticamente por el sistema operativo y no corresponden a acciones directas del usuario. Estos se filtran y se ignoran en el analisis:



NEGOTIATE, SESSION_SETUP, LOGOFF, TREE_CONNECT, TREE_DISCONNECT, ECHO, CANCEL, LOCK, IOCTL, CHANGE_NOTIFY, OPLOCK_BREAK, FLUSH.

2. Evolucion del analizador

El desarrollo del analizador ha pasado por cuatro iteraciones. Esta seccion resume las versiones anteriores (v1, v2, v3) y los problemas que motivaron el desarrollo de la v4. Los lectores interesados unicamente en la version actual pueden saltar directamente a la seccion 3.

2.1 v1: Agrupamiento por ventana temporal

El primer analizador ([src/analizador_smb2.py](#)) agrupaba los paquetes usando una **ventana temporal de 2 segundos**: si dos paquetes consecutivos del mismo FileID/TreeID tenian una diferencia menor a 2 segundos, se consideraban parte de la misma operacion.

Problemas detectados:

1. **Falsa separacion por asincronia**: En SMB2, el cliente puede enviar multiples solicitudes sin esperar respuesta. Esto provoca que paquetes de la misma operacion lleguen separados por mas de 2 segundos, fragmentando una operacion real en varias.
2. **Falsa fusion por solapamiento**: Si dos operaciones distintas ocurren en menos de 2 segundos (frecuente en sincronizacion con la nube), sus paquetes se fusionan en una sola operacion.
3. **Resultado**: 5.364 operaciones detectadas en la traza de usuario, muchas incorrectamente agrupadas. El umbral de 2 segundos era heuristico y no se adaptaba al comportamiento real del protocolo.

2.2 v2: Agrupamiento por FileID (CREATE+CLOSE)

El segundo analizador ([src/analizador_smb2_v2.py](#)) cambio la estrategia: en lugar de ventana temporal, agrupa por **FileID** usando el par CREATE+CLOSE como delimitador. Cada FileID se procesa de forma independiente, lo que resuelve el problema de la asincronia.

Problemas detectados:

1. **Operaciones multi-FileID no detectadas**: Muchas operaciones de usuario generan **multiples FileIDs** (uno por cada archivo involucrado). Por ejemplo, COMPRIMIR ARCHIVO usa 2 FileIDs, COPIAR CARPETA usa 2+, MODIFICAR ARCHIVO (editor) usa 2+. Al separar por FileID, estas operaciones se fragmentan en operaciones atómicas independientes.
2. **Reglas 8-15 inalcanzables**: Las reglas del clasificador que requieren **num_creates** ≥ 2 o **num_file_ids** ≥ 2 nunca se activan porque cada FileID se procesa por separado.
3. **FINDs sin FileID**: Los comandos **QUERY_DIRECTORY** no tienen FileID, por lo que se procesan aparte con ventana temporal, impidiendo asociarlos correctamente con la operacion que los origina.
4. **Resultados**: 4.793 operaciones atómicas en traza de usuario (99,1% clasificadas), 5.350 en traza de ransomware (78,5% clasificadas). Ninguna operacion compuesta detectada.

2.3 v3: Agrupamiento por ventana global

El tercer analizador ([src/analizador_smb2_v3.py](#)) fue un **experimento** para probar un enfoque radicalmente distinto: agrupar por ventana global sin separar por FileID. La idea era que el primer CREATE de la traza inicia una operacion, y todos los paquetes hasta el siguiente CLOSE (de cualquier FileID) pertenecen a esa operacion.

Problemas detectados:

1. **Solapamiento masivo de FileIDs:** En una traza real, los FileIDs de diferentes operaciones se solapan constantemente. Al no separar por FileID, todos los paquetes se mezclan en una unica operacion gigante.
2. **Resultados inutilizables:** Traza de usuario: solo 3 operaciones detectadas (una con 5.322 FileIDs). Traza de ransomware: 1 unica operacion con los 26.742 paquetes.
3. **Conclusion:** El enfoque de ventana global sin separacion por FileID no funciona.

2.4 Clasificador de segundo nivel

Antes de desarrollar la v4, se intento un enfoque intermedio: el **clasificador de segundo nivel** ([src/clasificador_segundo_nivel.py](#)). La idea era tomar las operaciones atomicas de v2 y fusionarlas en operaciones compuestas mediante un post-procesado.

Problemas detectados:

1. **Fusion limitada:** Solo se detectaron 16 operaciones MODIFICAR ARCHIVO en la traza de usuario (de 4.793 atomicas). El resto no se pudo fusionar por criterios demasiado restrictivos.
2. **Sin efecto en traza de ransomware:** 0 operaciones se fusionaron.
3. **Complejidad anadida:** Duplicaba la logica de agrupacion, haciendo el sistema mas dificil de mantener.
4. **Conclusion:** El enfoque de post-procesado no era suficiente. Se necesitaba un cambio en el algoritmo de agrupacion desde la raiz.

2.5 Resumen de problemas no resueltos

La siguiente tabla de resumen de problemas no resueltos se recoge en el **Anexo E**.

3. Analizador v4: Operaciones simples y compuestas

3.1 Arquitectura del pipeline

El pipeline de la v4 consta de 4 pasos:



Los detalles del diagrama se recogen en el **Anexo E**.

3.2 Estructuras de datos

La v4 introduce una nueva clase [OperacionCompuesta](#) que envuelve los paquetes de una operacion, ya sea simple (1 FileID) o compuesta (multiples FileIDs):

```
class OperacionCompuesta:
    id: str
    fileid_raiz: str
    paquetes: List[PaqueteSMB2]
    operaciones_atomicas: List[Operacion]
    es_simple: bool
    tipo_atmica: str
    tipo_compuesta: str
```

La clase [Operacion](#) existente se modificó para añadir el campo **id_compuesta**, que indica a qué operacion compuesta pertenece cada atomica. Sus atributos principales se detallan en el **Anexo E**.

3.3 Algoritmo de agrupacion jerarquica

El corazón de la v4 es la funcion [agrupar_por_operacion_v4\(\)](#). El algoritmo funciona asi:

Fase 1 — Pre-escaneo

Recorrer todos los paquetes para identificar qué **FileIDs** tienen al menos un **CLOSE** (**fids_con_close**).

Fase 2 — Agrupación jerárquica

Recorrer los paquetes en orden cronológico aplicando las siguientes reglas:

- Si el paquete **no tiene FileID** (**QUERY_DIRECTORY**, **QUERY_INFO**):
 - Si hay operación activa → integrarlo en ella
 - Si no → guardarlo para procesar al final por **TreeID**
- Si el paquete es un **CREATE**:

- Si su **FileID** no está en **fids_con_close** → descartar (CREATE huérfano)
- Si no hay operación activa → es el raíz, iniciar **OperacionCompuesta**
- Si ya hay operación activa:
 - Si es el mismo **FileID** que el raíz → cerrar la actual e iniciar nueva
 - Si no → es un subordinado, añadir a **fids_abiertos**
- Si el paquete es un **CLOSE**:
 - Si su **FileID** está en **fids_abiertos** → añadirlo y eliminarlo de **fids_abiertos**
 - Si es el **CLOSE** del raíz → cerrar la **OperacionCompuesta**
- Si el paquete es **cualquier otro comando** (**READ**, **WRITE**, **SET_INFO**, etc.):
 - Si su **FileID** está en **fids_abiertos** → añadirlo
 - Si no → descartar

Fase 3 — Cierre

Si queda una operación sin cerrar al final de la traza, se añade igualmente. Los paquetes sin **FileID** restantes se agrupan por **TreeID** con una ventana temporal de 2 segundos.

Ejemplo visual:

```
# Paquetes cronológicos:
[CREATE FID_A]      # raíz, inicia op_compuesta_1
[QUERY_DIRECTORY] # FIND sin FileID, se integra en op_compuesta_1
[READ FID_A]
[CREATE FID_B]      # subordinado, se añade a fids_abiertos
[SET_INFO FID_B]
[WRITE FID_B]
[CLOSE FID_B]      # se cierra FID_B, pero op_compuesta_1 sigue abierta
[READ FID_B]      # FID_B ya cerrado, pero estaba en fids_abiertos_original -> se mete
[CREATE FID_C]      # otro subordinado
[WRITE FID_C]
[CLOSE FID_C]
[CLOSE FID_A]      # se cierra el raíz -> op_compuesta_1 termina

# Resultado: 1 OperacionCompuesta con 3 FileIDs (FID_A, FID_B, FID_C)
```

3.4 Pre-escaneo de FileIDs con CLOSE

El pre-escaneo (`_pre_escanear_fids_con_close()`) es una fase previa al agrupamiento que identifica que FileIDs tienen al menos un CLOSE. Solo los CREATES con FileID que tiene CLOSE confirmado pueden ser raíz de una operación compuesta.

Motivación: Sin este pre-escaneo, el primer CREATE de la traza (que puede no tener CLOSE por ser una captura incompleta) se convierte en el "raíz" y captura todos los paquetes siguientes en una única operación gigante. Esto ocurrió durante el desarrollo: el primer CREATE de la traza de usuario tenía un FileID que nunca se cerraba, atrapando 25.580 paquetes en una sola operación.

Resultado del pre-escaneo en trazas reales:

Traza	Paquetes totales	FileIDs con CREATE	FileIDs con CLOSE	Descartados	% descarte
Traza_user_5.csv	26.817	4.760	4.752	8	0,17%
salidaSemana.txt	17.180.581	3.294.444	3.205.546	88.903	2,70%

3.5 Extraccion de operaciones atomicas

La funcion `extraer_operaciones_atomicas()` toma los paquetes de una operacion compuesta y los separa por FileID, creando una **Operacion** por cada uno. Cada operacion atomica se clasifica con el clasificador clasico (`clasificar_operacion()`), que aplica las 21 reglas en cascada.

3.6 Clasificador de operaciones compuestas

La función `clasificar_operacion_compuesta()` opera directamente sobre los paquetes de la operación y solo se aplica cuando esta involucra 2 o más FileIDs. Implementa 7 reglas multi-FileID que cubren las operaciones compuestas más habituales: subida y bajada de carpetas, copia, compresión de archivo y carpeta, modificación con editor, y operación compleja con modificación masiva. Si ninguna regla coincide, la operación se clasifica como OPERACIÓN COMPLEJA genérica. La descripción detallada de cada regla y sus condiciones se recoge en el **Anexo E**.

3.7 Integracion de FINDs en operaciones activas

En v2, los comandos **QUERY_DIRECTORY** (FIND) no tenian FileID y se procesaban aparte con ventana temporal. En v4, si un FIND aparece dentro del rango de una operacion activa (entre su CREATE raiz y su CLOSE raiz), se integra en ella.

Resultados de integracion:

- **Traza de usuario:** 633 paquetes sin FileID integrados (100% de los QUERY_DIRECTORY dentro de operaciones activas)
- **Traza de salidaSemana.txt:** 2.572.008 paquetes sin FileID integrados (869.750 QUERY_DIRECTORY + 805.182 QUERY_INFO + otros)

3.8 Soporte multi-formato (CSV y TXT)

El proyecto trabaja con dos formatos de entrada diferentes:

1. **CSV de Wireshark** (formato tabla exportado): usado para *Traza_user_5.csv* y *Traza_ransom_formato_tabla.csv*
2. **TXT de salidaSemana:** un formato espacio-separado con 20+ columnas, generado por una herramienta de captura diferente

Cada formato tiene distinto separador, número de columnas, codificación y cabeceras. La funcion `_detectar_formato()` analiza las primeras líneas del archivo para determinar el formato automaticamente.

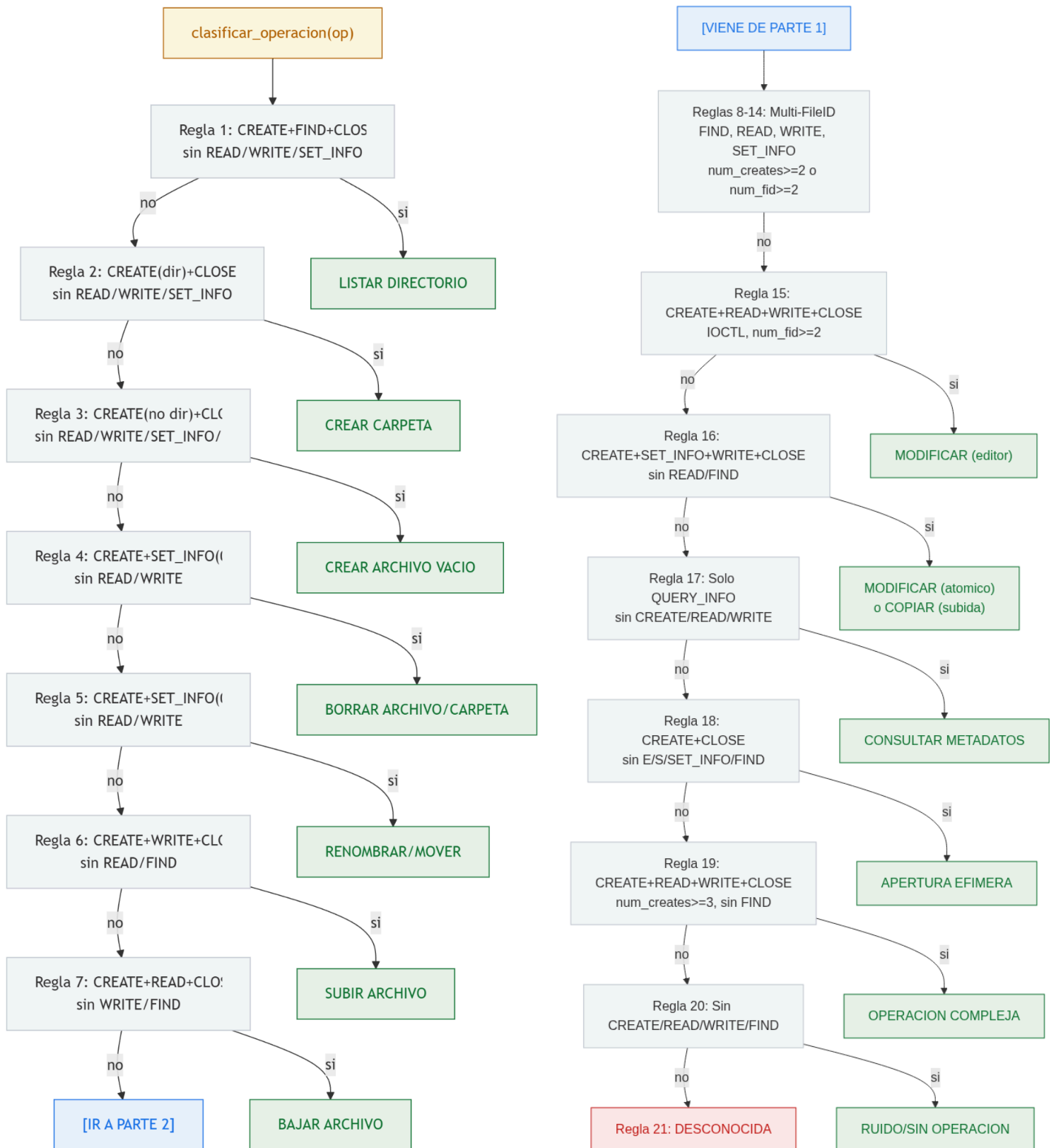
4. Clasificación de operaciones

4.1 Reglas de clasificación atómica (21 reglas)

Cada operación atómica agrupada por FileID se clasifica según los comandos SMB2 que contiene. Las reglas se aplican en **orden de prioridad** (de más específica a más genérica). La primera que cumple las condiciones gana.

1. **Operaciones de navegación y creación (Reglas 1-3):** LISTAR DIRECTORIO, CREAR CARPETA, CREAR ARCHIVO VACÍO
2. **Operaciones de modificación de metadatos (Reglas 4-5):** BORRAR, RENOMBRAR/MOVER
3. **Operaciones de transferencia (Reglas 6-7):** SUBIR ARCHIVO, BAJAR ARCHIVO
4. **Operaciones multi-FileID (Reglas 8-15):** SUBIR/BAJAR/COPIAR CARPETA, COMPRIMIR, MODIFICAR con editor
5. **Operaciones residuales (Reglas 16-21):** MODIFICACIÓN ATÓMICA, CONSULTAR METADATOS, APERTURA EFÍMERA, OPERACIÓN COMPLEJA, RUIDO, DESCONOCIDA

4.2 Arbol de decision



4.3 Análisis de solapamientos entre reglas

El clasificador evalúa las reglas en orden secuencial, de la más específica a la más genérica. Cuando una regla coincide, devuelve ese resultado y no sigue evaluando. Esto significa que el orden de las reglas es crítico y puede provocar que algunas operaciones sean "capturadas" por una regla que no les corresponde si comparten la misma estructura básica de comandos.

Sin embargo, la arquitectura de v4 mitiga este problema mediante dos niveles de clasificación:

- **Clasificador atómico (Reglas 1-21):** opera sobre operaciones de un solo FileID. Al estar cada operación atómica aislada por FileID, los solapamientos que antes ocurrían al mezclar múltiples FileIDs en una misma operación ya no se producen.
- **Clasificador de compuestas (7 reglas multi-FileID):** opera sobre los paquetes raw de la operación completa, permitiendo distinguir patrones que requieren varios FileIDs (COMPRIMIR, COPIAR CARPETA, MODIFICAR).

Los casos que persisten, tanto los resueltos por esta arquitectura como los inherentes al protocolo SMB2 y el solapamiento conocido entre COMPRIMIR CARPETA y COPIAR CARPETA, se documentan en detalle en el **Anexo E**.

Impacto en resultados reales: De las 23 operaciones de prueba, 22 se clasifican correctamente. El único fallo es COMPRIMIR CARPETA → COPIAR CARPETA, que corresponde al solapamiento documentado en el clasificador de compuestas. En trazas reales, este solapamiento no afecta significativamente al análisis global, ya que ambas operaciones implican lectura y escritura de datos.

5. Problemas encontrados durante el desarrollo

5.1 El problema de la asincronía

5.1.1 Naturaleza asíncrona de SMB2

SMB2 permite que el cliente envíe múltiples solicitudes sin esperar la respuesta de cada una. Esto significa que:

- Los comandos pueden solaparse temporalmente: un **CREATE** para un archivo puede enviarse antes de que llegue el **CLOSE** del archivo anterior
- El orden de llegada de los paquetes en la traza no refleja necesariamente el orden lógico de las operaciones
- Los **CLOSE** pueden llegar desordenados: el cierre de un archivo interno puede registrarse después del cierre de la carpeta que lo contiene

5.1.2 Cómo lo resuelve la v4

La v4 maneja la asincronía mediante el algoritmo de agrupación jerárquica con pre-escaneo de **FileIDs**, descrito en la sección 3.3. A diferencia de las versiones anteriores, que usaban ventanas temporales como heurístico, este enfoque toma el **FileID** como única referencia fiable para separar operaciones, eliminando la necesidad de cualquier umbral de tiempo. Esto permite reconstruir correctamente las operaciones **multi-FileID** incluso cuando sus paquetes aparecen entremezclados en la traza.

5.2 CREATEs huérfanos sin CLOSE

Al ejecutar la v4 por primera vez sobre la traza de usuario (26.817 paquetes), solo se detectaron 10 operaciones compuestas. La causa fue que el primer **CREATE** de la traza tenía un **FileID** que nunca se cerraba, por lo que el algoritmo lo tomó como raíz y capturó todos los paquetes siguientes en una única operación gigante (25.580 paquetes, 4.758 **FileIDs**).

La solución fue añadir la fase de pre-escaneo `_pre_escanear_fids_con_close()`, que identifica qué **FileIDs** tienen al menos un **CLOSE** antes del agrupamiento. Solo los **CREATEs** con **CLOSE** confirmado pueden ser raíz de una **OperacionCompuesta**.

En la traza de **salidaSemana.txt** (17,2M paquetes), el pre-escaneo descartó 88.903 **FileIDs** (2,70% del total). El análisis reveló que el 100% de estos **FileIDs** sin **CLOSE** se concentran en el último día de captura, lo que apunta a una captura incompleta como causa: el **CLOSE** de estos handles ocurrió después de que terminara la ventana de registro. El porcentaje de descarte es aceptable y no afecta significativamente a los resultados. El detalle completo del análisis se recoge en el **Anexo E**.

5.3 Integración de FINDs en operaciones activas

Al ejecutar la v4 sobre trazas reales se observó que ningún comando **QUERY_DIRECTORY** (FIND) se integraba en las operaciones activas, lo que impedía clasificar correctamente operaciones como SUBIR CARPETA, BAJAR CARPETA o COPIAR CARPETA. La causa dependía del formato de la traza.

En el formato TXT, Wireshark exporta los FINDs con un **FileID** de valor todo ceros (**00000000...**). El lector interpretaba ese GUID como un **FileID** válido, por lo que el paquete nunca entraba en la condición de integración y se descartaba. La solución fue añadir una comprobación explícita para tratar ese GUID como ausencia de **FileID**. En el formato CSV el problema no existía: los FINDs venían con la columna vacía y se integraban correctamente cuando había una operación activa. Los que aparecían fuera del rango de cualquier operación se procesan aparte por **TreeID**, que es el comportamiento esperado.

Tras la corrección, **2.572.008 paquetes** de **salidaSemana.txt** sin **FileID** se integraron correctamente en operaciones activas, con el 100% de los **QUERY_DIRECTORY** dentro del rango de una operación activa integrados en ambos formatos.

5.4 Multi-formato de entrada

Contexto: El proyecto trabaja con dos formatos de entrada diferentes:

1. **CSV de Wireshark** (formato tabla exportado): usado para *Traza_user_5.csv* y *Traza_ransom_formato_tabla.csv*
2. **TXT de salidaSemana**: un formato espacio-separado con 20+ columnas, generado por una herramienta de captura diferente

Cada formato tiene distinto separador, número de columnas, codificación y cabeceras.

Solucion: Se implemento una funcion `_detectar_formato()` que analiza las primeras líneas del archivo para determinar el formato automaticamente. Ver sección 3.8 para más detalles.

6. Validacion y pruebas

6.1 Estrategia de validacion

Para garantizar la correccion del analizador v4, se implementaron **tres niveles de validacion**:

1. **Tests automatizados con pytest** (22 tests): Verifican componentes ortogonales del pipeline que no estan cubiertos por los validators.
2. **Validador de todas las operaciones** (23 casos): Verifica que cada tipo de operacion se clasifica correctamente de forma aislada.
3. **Validador de pipeline completo** (7 casos entremezclados): Verifica el pipeline completo con operaciones entrelazadas como en trafico real.

6.2 Tests automatizados con pytest

Se eligio **pytest** como framework de testing.

Los tests se organizan en el archivo [tests/test_integracion_v4.py](#) y cubren seis grupos: pre-escaneo de **FileIDs**, agrupación jerárquica, extracción de operaciones atómicas, clasificación de compuestas, generación de reporte y lectura de CSV real. El resultado es 22/22 tests correctos. La tabla de grupos con el detalle de qué verifica cada uno, junto con la captura de terminal de la ejecución, se recogen en el **Anexo E**.

```
tests/
__init__.py          # Paquete Python
helpers.py           # Funcion pkt() compartida
test_integracion_v4.py      # Tests de integracion v4 (22 tests)
validar_todas_operaciones.py # Validador de todas las operaciones (23 casos)
validar_pipeline_v4.py     # Validador de pipeline completo (7 casos)
```

Ejecucion: `python -m pytest tests/test_integracion_v4.py -v`

6.3 Validador de todas las operaciones

El script [tests/validar_todas_operaciones.py](#) construye manualmente una operacion de cada tipo definido en la clasificacion, con la secuencia exacta de comandos SMB2 que las caracteriza. Cada operacion se pasa directamente a [clasificar_operacion\(\)](#) para verificar que el clasificador la reconoce correctamente.

A diferencia de los tests unitarios de pytest, que verifican casos aislados, este script prueba **23 operaciones simultáneamente** en una sola ejecucion, lo que permite detectar problemas de solapamiento entre reglas. La tabla de resultados y la captura de terminal se recogen en el **Anexo E**.

Ejecucion: `python tests/validar_todas_operaciones.py`

6.4 Validador de pipeline completo

El script `tests/validar_pipeline_v4.py` construye casos de prueba con operaciones **entrelazadas** (como en tráfico real) y ejecuta el pipeline v4 completo: *agrupar* → *extraer atomicas* → *clasificar*. Esto verifica que el algoritmo de agrupación jerárquica funciona correctamente incluso cuando los paquetes de diferentes operaciones aparecen entremezclados. Los 7 casos de prueba cubren desde operaciones secuenciales simples hasta reutilización de **FileID** y **CREATEs** sin **CLOSE**. La tabla de casos con las operaciones esperadas y la captura de terminal se recogen en el **Anexo E**.

Ejecucion: `python tests/validar_pipeline_v4.py`

6.5 Resultados de validacion

Tests pytest: 22/22

Validador de todas las operaciones: 22/23 correctas (1 fallo esperado)

El unico fallo es **COMPRIMIR CARPETA**, que se clasifica como **COPIAR CARPETA** debido al orden de prioridad de las reglas: la Regla 12 (**COPIAR CARPETA**) se evalúa antes que la Regla 14 (**COMPRIMIR CARPETA**), y como la Regla 12 no comprueba la ausencia de **SET_INFO**, captura la operacion antes de que llegue a la Regla 14. Este es un solapamiento conocido y documentado (ver sección 4.5).

Validador de pipeline completo: 9/9 (7 casos, algunos con multiples operaciones internas)

7. Resultados con trazas reales

7.1 Traza de usuario (Traza_user_5.csv)

Configuracion: 26.817 paquetes SMB2, formato CSV de Wireshark. Traza de actividad de usuario real.

Resultados v4:

Tipo	Simple	Compuestas	Total
SUBIR ARCHIVO	27	-	27
BORRAR ARCHIVO	18	-	18
RENOMBRAR/MOVER ARCHIVO	9	-	9
CREAR CARPETA	2	-	2
COPIAR CARPETA	-	7	7
OPERACION COMPLEJA	-	9	9
SIN CLASIFICAR	5	-	5
Total	61	16	77

Interpretacion:

- La v4 detecta **77 operaciones** (61 simples + 16 compuestas).
- De esas 77 operaciones se extraen **4.752 operaciones atomicas** (99,1% de las 4.793 que v2 reportaba en la misma traza). La diferencia de 41 operaciones se explica por los **34 CREATEs sin CLOSE** que v2 contaba incorrectamente como operaciones y que v4 descarta mediante el pre-escaneo, más 7 operaciones que v2 generaba por FINDs procesados aparte con la ventana temporal.
- Cada operacion compuesta en v4 engloba multiples operaciones atomicas de v2. Por ejemplo, una operacion "COPIAR CARPETA" en v4 contiene decenas de SUBIR ARCHIVO, BORRAR ARCHIVO, etc. como sub-operaciones.
- Se detectan **16 operaciones compuestas** que v2 no podia detectar: 7 COPIAR CARPETA y 9 OPERACION COMPLEJA.
- Las 61 operaciones simples se clasifican correctamente como SUBIR, BORRAR, RENOMBRAR, CREAR CARPETA.
- 5 operaciones simples quedan SIN CLASIFICAR (no coinciden con ninguna regla atomica).

7.2 Traza de ransomware (Traza_ransom_formato_tabla.csv)

Configuracion: 26.742 paquetes SMB2, formato CSV de Wireshark. Traza de ransomware cifrando archivos en una carpeta compartida.

Resultados v4:

Tipo	Simple	Compuestas	Total
CREAR ARCHIVO VACIO	26	-	26
BORRAR ARCHIVO	16	-	16
RENOMBRAR/MOVER ARCHIVO	5	-	5
CREAR CARPETA	2	-	2
OPERACION COMPLEJA	-	16	16
Total	49	16	65

Interpretacion:

- La v4 detecta **65 operaciones** (49 simples + 16 compuestas).
- Las 16 operaciones compuestas se clasifican todas como OPERACION COMPLEJA (generico), lo que indica que el ransomware no sigue los patrones de usuario (COPIAR CARPETA, BAJAR CARPETA, etc.) que el clasificador de compuestas reconoce.
- Las 49 operaciones simples se clasifican correctamente como CREAR ARCHIVO VACIO, BORRAR, RENOMBRAR, CREAR CARPETA.

7.3 Traza grande de trazas reales (salidaSemana.txt)

Configuracion: 17.180.581 paquetes SMB2, formato TXT espacio-separado. Traza de una semana completa de actividad en un servidor de archivos.

Resultados v4:

Tipo	Simple	Compuestas	Total
CREAR ARCHIVO VACIO	6.099	-	6.099
DESCONOCIDA	2.872	-	2.872
SUBIR ARCHIVO	562	-	562
CREAR CARPETA	271	-	271

BAJAR ARCHIVO	38	-	38
OPERACION COMPLEJA	-	850	850
OPERACION COMPLEJA (modif masiva)	-	471	471
COPIAR CARPETA	-	138	138
BAJAR CARPETA	-	56	56
SUBIR CARPETA	-	8	8
Total	9.842	1.523	15.458

Interpretacion:

- Se detectan **15.458 operaciones** en total (9.842 simples + 1.523 compuestas).
- La operacion simple más frecuente es CREAR ARCHIVO VACIO (6.099, 39%).
- Hay 2.872 operaciones DESCONOCIDA (19%), que corresponden a patrones no reconocidos por las 21 reglas del clasificador atomico.
- Se detectan **1.523 operaciones compuestas**: 850 OPERACION COMPLEJA (generico), 471 OPERACION COMPLEJA (modif masiva), 138 COPIAR CARPETA, 56 BAJAR CARPETA, 8 SUBIR CARPETA.
- Se integraron **2.572.008 FINDs/QUERY_INFO** dentro de operaciones activas, lo que permite clasificar correctamente las operaciones que requieren FIND.

Desglose de operaciones atómicas extraídas:

De las 15.458 operaciones compuestas (simples + multi-FileID) se extraen **3.212.339 operaciones atómicas** (1 por cada FileID), distribuidas así:

Tipo de operacion atomica	Total
CREAR ARCHIVO VACIO	2.008.144
BAJAR ARCHIVO	499.187
DESCONOCIDA	291.553
CREAR CARPETA	253.142
SUBIR ARCHIVO	156.115
SIN CLASIFICAR	4.093
OPERACION COMPLEJA	105
Total	3.212.339

Este desglose muestra que la mayoría de las operaciones atómicas son CREAR ARCHIVO VACIO (2.008.144, 62,5%), seguidas de BAJAR ARCHIVO (499.187, 15,5%) y DESCONOCIDA (291.553, 9,1%). Las 105 operaciones de tipo OPERACION COMPLEJA (modif masiva) corresponden a las 471 operaciones compuestas del mismo tipo, que al extraer sus atómicas generan múltiples FileIDs de tipo DESCONOCIDA y solo 105 se clasifican como modif masiva.

5. CONCLUSIONES Y LÍNEAS FUTURAS

1. Conclusiones

1.1 Análisis Wireshark y operaciones de usuario

El trabajo ha demostrado que el protocolo SMB2 expone una semántica operativa suficientemente rica como para reconstruir, de forma pasiva y no intrusiva, la totalidad de las acciones que un usuario ejecuta sobre un sistema de archivos centralizado. El análisis de los campos de cabecera (CreateOptions, DesiredAccess, FileInformationClass y NT_Status) junto con el FileID como hilo conductor de 128 bits, permite distinguir sin ambigüedad entre operaciones que a nivel de usuario parecen similares pero que a nivel de protocolo generan secuencias de comandos atómicos completamente distintas: un renombrado se traduce en CREATE → SET_INFO → CLOSE, mientras que una subida de archivo requiere CREATE → WRITE → CLOSE, y una descarga invierte el sentido de la transferencia con CREATE → READ → CLOSE.

Un hallazgo relevante del proceso de análisis es la importancia del entorno experimental para obtener trazas representativas. El intento inicial con bucle local (127.0.0.1) produjo capturas artificiales en las que los paquetes READ y WRITE eran invisibles para Wireshark debido a la optimización por caché del sistema operativo, lo que habría invalidado cualquier modelo construido sobre esa base. La transición a un entorno distribuido real con dos equipos físicos conectados por Wi-Fi fue una decisión metodológica imprescindible, no opcional, que permitió observar el comportamiento real del protocolo y confirmar que la secuencia teórica documentada en MS-SMB2 se corresponde fielmente con el tráfico capturado.

El modelado mediante máquinas de estados finitas (FSM) ha resultado ser el enfoque correcto para abstraer las operaciones de usuario a partir de comandos atómicos dispersos. Sin embargo, la aplicación práctica ha evidenciado una limitación inherente al propio protocolo: SMB2 es asíncrono por diseño, y el cliente puede enviar múltiples solicitudes sin esperar respuesta, lo que hace que los paquetes de operaciones distintas aparezcan entremezclados en la traza. Este problema no puede resolverse únicamente con FSM clásicas sin un mecanismo previo de agrupación por FileID, lo que obligó a replantear la arquitectura del analizador en varias ocasiones antes de llegar a la solución definitiva.

El catálogo de operaciones definido (16 tipos entre simples y compuestas) cubre las acciones más frecuentes en un entorno corporativo, pero deja fuera casos menos habituales que generan patrones no reconocidos. En la traza de una semana real, el 19% de las operaciones atómicas quedaron

clasificadas como DESCONOCIDA, lo que indica que el espacio de comportamientos observables en un servidor de producción es más amplio de lo que un análisis controlado en laboratorio puede anticipar. Este resultado, lejos de ser un fracaso, delimita con precisión el trabajo que queda por hacer.

1.2 Desarrollo clasificador operaciones de usuario

El analizador ha alcanzado un nivel de madurez funcional claro, reflejado en los resultados de validación: 99,1% de clasificación correcta en la traza de usuario, 22/22 tests pytest, 22/23 operaciones de prueba correctamente clasificadas y 9/9 casos de pipeline completo superados. Estos números son sólidos para un prototipo de investigación, pero deben interpretarse con cautela: la traza de usuario es una captura controlada generada en condiciones de laboratorio, mientras que el comportamiento real en un servidor corporativo (representado por salidaSemana.txt con 17,18 millones de paquetes) produce un perfil de operaciones significativamente más heterogéneo.

La evolución iterativa a través de cuatro versiones del analizador ha sido necesaria y ha aportado un aprendizaje concreto sobre los límites de cada enfoque. La v1 demostró que la ventana temporal es un heurístico frágil cuando SMB2 opera de forma asíncrona. La v2 demostró que el agrupamiento por FileID individual resuelve la asincronía pero fragmenta las operaciones compuestas. La v3 demostró que un agrupamiento global sin separación por FileID es directamente inviable en trazas reales. La v4 combina lo mejor de los enfoques anteriores: agrupamiento jerárquico con FileID como referencia fiable, pre-escaneo para detectar handles huérfanos y clasificación en dos niveles (atómica y compuesta) que permite detectar las 16 operaciones multi-FileID que las versiones anteriores ignoraban por completo.

El único fallo de clasificación conocido y documentado (COMPRIMIR CARPETA clasificado como COPIAR CARPETA) es consecuencia directa del orden de evaluación de las reglas y no de un error en la lógica del clasificador. Ambas operaciones comparten el mismo esqueleto de comandos SMB2; la distinción entre ellas requiere criterios adicionales que el clasificador actual no implementa. En un contexto de análisis forense real, confundir estas dos operaciones tiene un impacto limitado porque ambas implican lectura y escritura de datos, pero en un sistema de detección de ransomware podría introducir falsos negativos si el malware utilizara la compresión como vector.

2. Limitaciones

2.1 Análisis SMB2 y operaciones de usuario

1. **Dependencia de SMB2 en entorno local:** La metodología cubre únicamente el escenario de almacenamiento centralizado en red local (NAS, servidores de archivos Windows). El tráfico cloud (OneDrive, Google Drive) viaja cifrado con TLS mediante protocolos propietarios no documentados, lo que hace inviable cualquier inspección externa sin ingeniería inversa. Este es el punto ciego que motivó el proyecto y que permanece sin resolver.
2. **Base de validación reducida:** La validación se realizó sobre tres trazas. Con una sola traza de ransomware de 26.742 paquetes no es posible afirmar que el perfil observado (creación masiva de archivos vacíos, borrado y renombrado sistemático) sea representativo de todas las familias de ransomware existentes. Diferentes familias pueden generar patrones de tráfico SMB2 distintos o imitar el comportamiento de un usuario legítimo para evadir la detección.
3. **Modelado sobre carpetas controladas:** Las operaciones recursivas están modeladas sobre directorios de contenido controlado. En producción, las operaciones sobre directorios con contenido masivo generan concatenaciones de FileIDs que el clasificador agrupa como OPERACION COMPLEJA genérica, con la consiguiente pérdida de granularidad analítica precisamente en las operaciones que más datos mueven.
4. **Ausencia de mecanismo de respuesta activa:** El trabajo concluye en la fase de generación de perfiles. La detección de un comportamiento anómalo no desencadena ninguna acción automática (bloqueo de sesión, alerta en tiempo real, cuarentena de IP), lo que limita su aplicabilidad directa como sistema de protección y lo sitúa exclusivamente en el ámbito del análisis forense.

2.2 Desarrollo clasificador operaciones de usuario

1. **Clasificador de compuestas limitado:** El 56% de las operaciones compuestas de la traza semanal (850 de 1.523) quedan sin patrón reconocido (OPERACION COMPLEJA genérica). El analizador detecta que algo relevante está ocurriendo pero no puede describir qué, lo que es la debilidad más significativa del sistema para un uso en producción.
2. **Reglas sin modelo formal:** Las reglas de clasificación están implementadas como condiciones en cascada en Python, sin representación formal de las FSM que las motivan.

Añadir una nueva regla requiere comprender el orden de prioridad de todas las existentes para evitar solapamientos, y el riesgo de introducir regresiones crece con cada adición.

3. **Doble pasada por el pre-escaneo:** El pre-escaneo de FileIDs con CLOSE requiere recorrer toda la traza dos veces, duplicando el tiempo de procesamiento. En trazas de 17 millones de paquetes esto tiene un impacto real, especialmente si se pretende extender la herramienta hacia un uso en tiempo casi real.
4. **FINDs fuera de operación activa:** Los comandos QUERY_DIRECTORY que aparecen fuera del rango de cualquier operación activa se siguen agrupando por TreeID con ventana temporal, igual que en v2. Esto puede dejar FINDs huérfanos sin asociar correctamente a ninguna operación.
5. **Soporte de formatos limitado:** El formato ODS, usado en algunas trazas, no se soporta directamente y requiere conversión manual previa a CSV, introduciendo un paso de preprocesamiento que puede generar errores o perderse en el flujo de trabajo.

3. Trabajo Futuro

3.1 Análisis Wireshark y operaciones de usuario

1. **Clasificador binario ransomware/usuario:** Usar el perfil de operaciones detectadas (tipos, frecuencias, volúmenes y cadencia temporal) como características para entrenar un clasificador supervisado que distinga tráfico legítimo de malicioso. La comparativa entre las trazas ya disponibles muestra diferencias cualitativas claras que constituyen una base de partida sólida.
2. **Detección en tiempo real:** Extender el analizador para operar sobre un flujo de captura continuo en lugar de sobre trazas ya capturadas. Esto requiere replantear el algoritmo de agrupación hacia un modelo incremental con ventana deslizante que no necesite acceso al futuro de la traza para identificar FileIDs con CLOSE.
3. **Ampliación de la base de trazas:** Validar el clasificador sobre capturas de diferentes entornos corporativos, distintas versiones de cliente Windows y diferentes familias de ransomware, para cuantificar la generalización real del sistema y detectar patrones que el laboratorio controlado no puede reproducir.
4. **Análisis de volumen:** Incorporar el tamaño de los datos transferidos (total_read, total_write) y la cadencia temporal entre operaciones como variables adicionales para el análisis de comportamiento, lo que permitiría distinguir entre operaciones sobre archivos pequeños y grandes y caracterizar el ritmo de actividad del usuario frente al de un proceso automatizado.

3.2 Desarrollo clasificador operaciones de usuario

1. **Refactorizar clasificadores a FSM:** Reemplazar las reglas en cascada por máquinas de estados finitos formales, lo que haría el código más legible, mantenible y verificable. Herramientas como la librería *transitions* de Python o tablas de transición explícitas son candidatos directos para esta refactorización.
2. **Diferenciación de usuarios concurrentes:** Las trazas analizadas en este trabajo se tratan como un flujo único de actividad sin distinguir qué usuario concreto hay detrás de cada operación. Una línea de trabajo relevante sería explotar los campos SessionID y TreeID del protocolo para aislar y perfilar de forma independiente la actividad de múltiples usuarios que operan simultáneamente sobre el mismo servidor, lo que permitiría detectar comportamientos anómalos de un usuario concreto sin que el tráfico legítimo del resto contamine el análisis.

3. **Ampliar reglas del clasificador de compuestas:** Analizar las 850 operaciones sin patrón reconocido de salidaSemana.txt para identificar subconjuntos recurrentes de comandos y definir nuevas reglas que reduzcan el porcentaje de OPERACION COMPLEJA genérica del 56% actual.
4. **Optimizar el pre-escaneo:** Sustituir la doble pasada por la traza por una estructura de datos auxiliar que mantenga el estado de los FileIDs vistos y actualice su clasificación conforme llegan los CLOSE, reduciendo el coste temporal del pipeline a la mitad para trazas de gran volumen.
5. **Soporte para ODS:** Añadir un lector nativo del formato OpenDocument Spreadsheet para eliminar la conversión manual previa y hacer la herramienta directamente utilizable con exportaciones de herramientas de captura que generan ese formato.
6. **Visualización temporal:** Generar gráficos con la distribución temporal de las operaciones detectadas para identificar patrones de comportamiento visualmente: ráfagas de actividad, períodos de inactividad, picos anómalos de creación o borrado que podrían correlacionarse con un ataque en curso.

6. BIBLIOGRAFÍA

- [1] Microsoft Corporation, «[MS-SMB2]: Server Message Block (SMB)». 13 de abril de 2026. [En línea]. Disponible en: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-smb2/
- [2] «[MS-ERREF]: Windows Error Codes». Accedido: 29 de abril de 2026. [En línea]. Disponible en: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-erref/1bc92ddf-b79e-413c-bbaa-99a5281a6c90
- [3] M. Joshi y T. H. Hadi, «A Review of Network Traffic Analysis and Prediction Techniques», arXiv.org. Accedido: 29 de abril de 2026. [En línea]. Disponible en: <https://arxiv.org/abs/1507.05722v2>
- [4] Wireshark Team, «Wireshark User's Guide». 2024. [En línea]. Disponible en: https://www.wireshark.org/docs/wsug_html/
- [5] robinharwood, «New-SmbShare (SmbShare)». Accedido: 25 de mayo de 2026. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/powershell/module/smbshare/new-smbshare?view=windowsserver2025-ps>
- [6] A. Espinal, V. S. Padilla, y Y. Cevallos, «Performance Evaluation of Multimedia Traffic in Cloud Storage Services over Wi-Fi and LTE Networks».
- [7] Microsoft Corporation, «[MS-FSA]: File System Algorithms». 2025. [En línea]. Disponible en: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-fsa/
- [8] Microsoft Corporation, «[MS-CFB]: Compound File Binary File Format». 2024. [En línea]. Disponible en: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-cfb/
- [9] J. Zhou, W. Fu, W. Hu, Z. Sun, T. He, y Z. Zhang, «Challenges and Advances in Analyzing TLS 1.3-Encrypted Traffic: A Comprehensive Survey», *Electronics*, vol. 13, n.º 20, p. 4000, ene. 2024, doi: 10.3390/electronics13204000.
- [10] Y. Cui, Z. Lai, X. Wang, N. Dai, y C. Miao, «QuickSync: Improving Synchronization Efficiency for Mobile Cloud Storage Services», en *Proceedings of the 21st Annual International Conference on Mobile Computing and Networking*, en MobiCom '15. New York, NY, USA: Association for Computing Machinery, sep. 2015, pp. 592-603. doi: 10.1145/2789168.2790094.
- [11] «[MS-FSCC]: File System Control Codes». Accedido: 30 de abril de 2026. [En línea]. Disponible en: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-fscc/efbfe127-73ad-4140-9967-ec6500e66d5e
- [12] sdwheeler, «New-PSDrive (Microsoft.PowerShell.Management) - PowerShell». Accedido: 25 de mayo de 2026. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.management/new-psdrive?view=powershell-7.6>
- [13] Mario Vicente Obeso, «smb2-cloud-traffic-analyzer: Analizador de tráfico SMB2 y flujos de almacenamiento asíncronos», *GitHub Repository*, mayo de 2026. [En línea]. Disponible en: <https://github.com/vicente154000/smb2-cloud-traffic-analyzer>

ANEXOS

Esta sección contiene el material complementario, la documentación técnica y el código fuente desarrollados para dar soporte experimental y metodológico a este Trabajo Fin de Grado. Los anexos se estructuran de forma cronológica conforme al despliegue y desarrollo del proyecto.

Anexo A. Catálogo del Protocolo SMB2

- **A.1. Comparativa de Soluciones:** Auditoría basada en host vs. basada en red.
- **A.2. Comandos de Gestión:** Infraestructura y control de conexión (*Negotiate, Session Setup, Tree Connect*).
- **A.3. Comandos Atómicos:** Diccionario de operaciones base (*Create, Read, Write, Close*).
- **A.4. Atributos Técnicos:** Campos de control clave (*CreateOptions, DesiredAccess, Status*).

Anexo B. Fundamentos Técnicos del Análisis de Trazas

- **B.1. Mecanismos de Correlación:** Uso de *Message ID* y *File ID (GUID)* como punteros lógicos.
- **B.2. Lógica de Filtrado en Wireshark:** Criterios de discriminación, validación de estado y descarte de tráfico redundante.

Anexo C. Capturas de Tráfico en Wireshark

- **C.1. Trazas de Directorios:** Evidencias de creación, listado, renombrado, movimiento y borrado.
- **C.2. Trazas de Archivos:** Evidencias de subida, descarga, modificación, copia, compresión y eliminación.

Anexo D. Patrones y Secuencias de Operaciones de Usuario

- **D.1. Patrones de Transferencia y Modificación:** Flujos lógicos de subida, descarga y edición de datos.
- **D.2. Metodología de Validación:** Uso de unidad remota para mitigar el efecto de la caché local.
- **D.3. Patrones de Organización:** Secuencias de comandos para renombrado, borrado, listado y copia.

Anexo E. Arquitectura del Software y Pruebas del Pipeline

- **E.1. Estructura de Datos:** Formato del CSV de entrada y definición del objeto *OperacionCompuesta*.
- **E.2. Matriz de Clasificación:** Solapamientos semánticos y gestión de anomalías (*CREATEs* huérfanos).
- **E.3. Batería de Pruebas:** Resultados de tests de integración con *pytest* y validación del *pipeline*.

AGRADECIMIENTOS

Dicen que el tiempo vuela, pero cuando miro atrás y pienso en el primer día que entré en la universidad, me doy cuenta de todo lo que cabe en cuatro años. Parecía que este día no iba a llegar, pero tras muchos cafés, noches apurando el sueño y problemas que parecían no tener solución, el camino llega a su fin.

Pero, sobre todo, han sido cuatro años de aprendizaje personal, donde no solo he aprendido a tirar líneas de código, organizar proyectos o resolver problemas técnicos, sino también a lidiar con la frustración, a tener paciencia y a darme cuenta de que los límites que creía tener solo estaban en mi cabeza.

Detrás de cada día de estudio, de cada época de exámenes y de cada hora de esfuerzo, hay personas que me han sostenido cuando las fuerzas faltaban. A ellos, que han sido mi verdadera motivación durante estos cuatro años, les dedico este trabajo.

A mi tutor, por todas las reuniones que hemos tenido, por su paciencia, su guía y por orientarme en este proyecto de la manera tan buena en la que lo ha hecho.

A mis amigos de la uni, que han hecho de estos años una experiencia mucho más amena. Por todas esas horas en la biblioteca, en la cafetería, de fiesta, de proyectos...y por demostrarme que las asignaturas imposibles se pasan mejor en buena compañía.

A mi familia; *mis padres*, que desde el principio han confiado ciegamente en mí y han apostado todo para que lo consiga; *mi hermana*, que desde pequeños hemos vivido todo juntos y aunque a veces parezca que no, ha estado a mi lado en cada momento; *mis abuelas*, que siempre me han apoyado, animado y estado orgullosas, contándole a todo el mundo lo bien que me iba; y a *mi novia*, que se ha comido todas mis semanas de estrés, mis chapas y mis tonterías y, aun así, siempre me ha conseguido sacar una sonrisa.

A mi cuadrilla, que a pesar de lo cabrones que son todos, llevan conmigo toda la vida. Por todas las cosas que hemos vivido juntos, los viajes, las fiestas, las historias que no se pueden contar y las que seguro están por venir, han estado en los mejores y en los peores momentos, y espero que así siga siendo durante mucho tiempo más.

Muchas gracias a todos.